

REMERCIEMENT



TABLE DES MATIÈRES

LISTE DES FIGURES	1
1 ÉTUDE GÉNÉRALE	2
1.1 INTRODUCTION	3
1.2 Présentation de l'organisme d'accueil	3
1.2.1 Présentation générale	3
1.2.2 Mission et vision	4
1.2.3 Valeurs et culture d'entreprise	4
1.2.4 Domaines d'expertise	4
1.3 Cadre général du projet	5
1.4 Contexte et problématique	5
1.4.1 Contexte des cours particuliers en Tunisie	5
1.4.2 Limites du processus actuel	6
1.4.3 Problématique	6
1.5 Étude de l'existant	7
1.5.1 Analyse et critique de l'existant	7
1.5.2 Limites des solutions existantes	8
1.6 Solution proposée	8
1.7 Méthodologie de travail	10
1.7.1 Cadre de gestion de projet agile (Scrum)	10
1.7.2 Aperçu du projet	11
1.7.3 Planification des Sprints pour EduMatch	13
1.7.4 Cycle de vie de l'IA inspiré de CRISP-DM	15
1.7.5 Outils et bonnes pratiques	17
1.8 Conclusion	17
2 Concepts Fondamentaux	18
2.1 INTRODUCTION	19
2.2 Intelligence Artificielle	19

2.2.1	Apprentissage Automatique (Machine Learning)	19
2.2.2	Deep Learning	20
2.3	Traitement Automatique du Langage Naturel (NLP)	21
2.3.1	Définition	21
2.3.2	Applications	21
2.3.3	Défis et Limites	22
2.4	Grands Modèles de Langage (LLM)	22
2.4.1	Définition	22
2.4.2	Caractéristiques des LLM	23
2.4.3	Principaux modèles LLM	23
2.4.4	Architecture des LLM	24
2.5	Optimisation des LLM pré-entraînés	25
2.5.1	Fine-tuning	25
2.5.2	RAG (Retrieval-Augmented Generation)	25
2.5.3	Prompt Engineering	26
2.5.4	Comparaison et choix pour EduMatch	26
2.6	Systèmes de Recommandation et Matching	27
2.6.1	Définition	27
2.6.2	Types de systèmes de recommandation	27
2.6.3	Algorithme de Matching d'EduMatch	28
2.7	Agents Conversationnels (Chatbots)	29
2.7.1	Définition et types	29
2.7.2	Architecture d'un chatbot à flux guidé hybride	29
2.8	Conclusion	30
3	Concepts Fondamentaux	31
3.1	Introduction	32
3.2	Intelligence Artificielle et NLP	32
3.3	Modèles de Langage (LLM)	32
3.4	Prompt Engineering	33
3.5	Agents Conversationnels (Chatbot)	33
3.6	Système de Recommandation et Matching	34
3.7	Conclusion	34
4	Analyse, Conception et Architecture du Système	35
4.1	Introduction	36

4.2	Étude des besoins	36
4.2.1	Identification des acteurs	36
4.2.2	Besoins fonctionnels	37
4.2.3	Besoins non fonctionnels	38
4.3	Conception UML	39
4.3.1	Présentation de l’UML	39
4.3.2	Diagramme des cas d’utilisation	39
4.3.2.1	Définition	39
4.3.2.2	Composition du diagramme	40
4.3.2.3	Diagramme global d’EduMatch	40
4.3.3	Diagramme d’activité	42
4.3.3.1	Définition	42
4.3.3.2	Flux de recherche via EduBot	43
4.3.4	Diagramme de séquence	44
4.3.4.1	Définition	44
4.3.4.2	Processus de réservation d’une séance	44
4.3.5	Diagramme de classes	45
4.3.5.1	Définition	45
4.3.5.2	Composition d’un diagramme de classes	46
4.3.5.3	Diagramme de classes d’EduMatch	46
4.4	Architecture Technique	47
4.4.1	Architecture 3-tiers	47
4.4.2	Architecture du chatbot EduBot	49
4.4.3	Workflow de validation des formateurs	50
4.5	Conclusion	4
5	Réalisation, justification des choix et implémentation	8
5.1	Introduction	10
5.2	Environnement de développement et justification des choix technologiques . .	10
5.2.1	Environnement de développement	10
5.2.2	Choix du modèle LLM — LLaMA 3.1 via l’API Groq	11
5.2.3	Choix de la stratégie d’optimisation IA — Prompt Engineering	12
5.2.4	Choix du framework backend — FastAPI	13
5.2.5	Choix du framework frontend — React avec Vite	14
5.2.6	Choix de la base de données — PostgreSQL	15
5.2.7	Choix de l’algorithme de matching	15

5.3	Présentation des interfaces réalisées	16
5.3.1	Page d'accueil et authentification	16
5.3.2	Espace étudiant	17
5.3.3	Espace formateur	18
5.3.4	Chatbot EduBot	19
5.3.5	Espace administrateur	20
5.4	Implémentation des composants clés	21
5.4.1	Algorithme de matching multicritères	21
5.4.2	Pipeline conversationnel d'EduBot	22
5.4.3	Gestion de l'authentification et des autorisations	23
5.4.4	Composant de recherche et affichage des résultats	23
5.5	Tests et validation	24
5.5.1	Tests unitaires et d'intégration	24
5.5.2	Tests fonctionnels	25



LISTE DES FIGURES

1.1	Logo 21C'Digital	3
1.2	Processus Scrum pour EduMatch	11
4.1	Diagramme de cas d'utilisation global — EduMatch	41
4.2	Diagramme d'activité — Flux de recherche via EduBot	43
4.3	Diagramme de séquence — Réservation d'une séance	45
4.4	Diagramme de classes complet — EduMatch	47
4.5	Architecture 3-tiers d'EduMatch	48
4.6	Pipeline de traitement d'EduBot	49
5.1	Page d'accueil d'EduMatch	17

ÉTUDE GÉNÉRALE

Sommaire

1.1 INTRODUCTION	3
1.2 Présentation de l'organisme d'accueil	3
1.2.1 Présentation générale	3
1.2.2 Mission et vision	4
1.2.3 Valeurs et culture d'entreprise	4
1.2.4 Domaines d'expertise	4
1.3 Cadre général du projet	5
1.4 Contexte et problématique	5
1.4.1 Contexte des cours particuliers en Tunisie	5
1.4.2 Limites du processus actuel	6
1.4.3 Problématique	6
1.5 Étude de l'existant	7
1.5.1 Analyse et critique de l'existant	7
1.5.2 Limites des solutions existantes	8
1.6 Solution proposée	8
1.7 Méthodologie de travail	10
1.7.1 Cadre de gestion de projet agile (Scrum)	10
1.7.2 Aperçu du projet	11
1.7.3 Planification des Sprints pour EduMatch	13
1.7.4 Cycle de vie de l'IA inspiré de CRISP-DM	15
1.7.5 Outils et bonnes pratiques	17
1.8 Conclusion	17

1.1 INTRODUCTION

Dans ce chapitre, nous allons placer notre projet dans son contexte général. Nous commençons par présenter l'entreprise d'accueil. Ensuite, nous consacrerons le reste de ce chapitre à l'étude critique de l'existant et à la problématique spécifique à notre projet, afin de proposer une solution.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation générale

21C'Digital est une agence digitale moderne basée à Sfax, spécialisée dans la création et le développement de solutions numériques innovantes. Depuis sa fondation en 2020, l'entreprise accompagne particuliers, entreprises et institutions publiques dans leur transformation digitale. Son ambition est de transformer les idées en réussites digitales grâce à la combinaison de créativité, de technologie et de performance.

Bien que basée à Sfax, 21C'Digital déploie ses activités à l'échelle mondiale, avec des projets réalisés en France, en Suisse et en Australie, ce qui témoigne de son expertise internationale et de sa capacité à collaborer sur des projets multiculturels.

La figure 1.1 montre le logo de l'entreprise 21C'Digital.



FIGURE 1.1 – Logo 21C'Digital

1.2.2 Mission et vision

La mission principale de 21C'Digital est de concevoir des solutions digitales sur mesure répondant aux besoins réels de ses clients, tout en favorisant leur croissance et leur autonomie numérique. Sa vision est de « construire l'avenir digital », en plaçant l'innovation au cœur de chaque projet et en contribuant à un écosystème technologique performant et durable.

1.2.3 Valeurs et culture d'entreprise

L'entreprise repose sur des valeurs fondamentales qui structurent son fonctionnement :

- **Innovation** : moteur de chaque projet entrepris
- **Excellence** : recherche de qualité et de performance mesurable
- **Engagement client** : accompagnement personnalisé et support réactif 24/7
- **Esprit d'équipe** : collaboration, apprentissage continu et créativité collective

1.2.4 Domaines d'expertise

L'agence couvre un large éventail de services dans le domaine du numérique :

- **Développement Web** : sites vitrines, e-commerce, plateformes SaaS
- **Développement Mobile** : applications Android et iOS personnalisées
- **Référencement SEO/SEA** et stratégie de visibilité en ligne
- **Design UX/UI** et conception d'expériences utilisateurs modernes
- **Marketing digital** : gestion de contenu, publicité en ligne, social media
- **Business Intelligence (BI)** et tableaux de bord interactifs
- **Intelligence Artificielle (AI)** pour des solutions intelligentes et prédictives
- **Intégration et formation** pour accompagner la digitalisation complète des entreprises

1.3 Cadre général du projet

Le projet intitulé « EduMatch — Plateforme intelligente de mise en relation apprenants-formateurs » s'inscrit dans le cadre du projet de fin d'études (PFE) pour l'obtention de la licence en Informatique à l'Institut International de Technologie (IIT).

Ce projet a été réalisé sur une période de quatre mois, allant de février à juin 2026. Il a pour objectif principal de concevoir et développer une plateforme web permettant de faciliter la mise en relation entre des apprenants (élèves, étudiants ou professionnels en reconversion) et des formateurs qualifiés dans divers domaines académiques et professionnels.

La plateforme vise à simplifier le processus de recherche, de recommandation et de réservation, tout en offrant une expérience utilisateur moderne et intuitive grâce à l'intégration de technologies web récentes et de solutions intelligentes.

1.4 Contexte et problématique

Cette section plante le cadre général de notre étude. Nous y décrivons d'abord l'importance des cours particuliers en Tunisie, puis nous identifions les lacunes du processus actuel avant de formuler la problématique centrale de notre projet.

1.4.1 Contexte des cours particuliers en Tunisie

En Tunisie, le recours aux cours particuliers est devenu un phénomène massif et structuré. Selon plusieurs études récentes, près de 75 % des familles y ont recours, notamment à l'approche des examens nationaux tels que la 9^e année et le baccalauréat.

Ce marché représente un volume économique important, estimé à plus de 3 milliards de dinars, et constitue une charge financière considérable pour les familles, pouvant atteindre entre 1000 et 2000 dinars par élève en période de révision.

Par ailleurs, la demande en soutien scolaire ne cesse d'augmenter, en raison des difficultés académiques rencontrées par les élèves et de la pression liée aux performances scolaires.

1.4.2 Limites du processus actuel

Malgré cette forte demande, le processus de recherche de cours particuliers reste traditionnel et peu structuré. Les principaux problèmes identifiés sont les suivants :

- **Absence de plateforme centralisée** : la recherche repose principalement sur le bouche-à-oreille, les groupes Facebook ou les petites annonces, sans système organisé.
- **Manque de filtrage et de personnalisation** : il est difficile de trouver un professeur correspondant précisément aux critères de l'apprenant (matière, niveau, budget, localisation, disponibilité).
- **Processus inefficace** : la recherche est souvent longue, incertaine et peut conduire à des choix non optimaux ce qui entraîne des pertes de temps et d'argent.
- **Inégalités d'accès** : les apprenants issus de milieux favorisés ont un accès plus facile à des enseignants qualifiés.

1.4.3 Problématique

Face à ces constats, il apparaît clairement qu'il n'existe pas en Tunisie de solution numérique centralisée, intelligente et efficace permettant de faciliter la mise en relation entre apprenants et formateurs.

Ainsi, la problématique principale de ce projet est formulée comme suit :

Comment concevoir une plateforme intelligente permettant d'optimiser la mise en relation entre apprenants et formateurs en Tunisie, en proposant un matching personnalisé, fiable et efficace, basé sur les technologies web modernes et l'intelligence artificielle ?

1.5 Étude de l'existant

Afin de positionner notre solution par rapport aux offres actuelles, cette section propose une analyse critique des plateformes de soutien scolaire existantes, aussi bien tunisiennes qu'internationales. Nous identifions ensuite leurs principales limites afin de justifier les apports de notre proposition.

1.5.1 Analyse et critique de l'existant

- **Plateformes tunisiennes :**

Plusieurs plateformes de soutien scolaire existent sur le marché tunisien. Le tableau 1.1 présente les principales, accompagnées de leurs limites.

Plateforme	Description	Limites
Mathezer	Cours vidéo et sessions interactives en mathématiques pour collège et lycée	Spécialisée uniquement en mathématiques
Thabet Academy	Cours vidéo et classes virtuelles pour primaire, collège et secondaire	Absence de recommandation intelligente
Tadris.tn	Plateforme de mise en relation pour cours particuliers en ligne	Manque d'outils analytiques avancés

TABLE 1.1 – Plateformes tunisiennes de soutien scolaire

- **Plateformes internationales**

À l'échelle internationale, plusieurs plateformes de mise en relation entre élèves et professeurs particuliers dominent le marché. Le tableau 1.2 en dresse une sélection représentative.

Plateforme	Description	Limites
Superprof	Large choix de disciplines, profils détaillés et avis utilisateurs	Absence de chatbot intelligent et de score de compatibilité
Preply	Plateforme spécialisée dans l'apprentissage des langues avec classe virtuelle	Limitée principalement aux langues
Wyzant	Large catalogue de matières avec réservation directe	Orientée marché international, sans personnalisation avancée

TABLE 1.2 – Plateformes internationales de cours particuliers

1.5.2 Limites des solutions existantes

Malgré la diversité des plateformes disponibles, plusieurs limites communes peuvent être identifiées :

- Absence de chatbot intelligent pour accompagner l'utilisateur
- Manque de système de recommandation basé sur un score de compatibilité
- Faible exploitation des données et absence de tableaux de bord analytiques
- Personnalisation limitée de l'expérience utilisateur

1.6 Solution proposée

Le projet EduMatch propose une solution innovante de mise en relation entre étudiants et professeurs en Tunisie. Le tableau 1.3 résume les principales innovations apportées par rapport aux solutions existantes.

Afin de répondre aux limites identifiées, le projet EduMatch propose plusieurs innovations :

Innovation	Description
Chatbot IA conversationnel	Accompagne l'étudiant dans sa recherche et facilite la réservation
Matching intelligent	Prend en compte plusieurs critères (niveau, matière, budget, disponibilité)
Score de compatibilité	Classe les professeurs selon leur adéquation avec le profil de l'étudiant
Tableau de bord analytique	Analyse des données (réservations, tendances, performances)
Réservation flexible	Gestion des cours en ligne et en présentiel avec notifications

TABLE 1.3 – Innovations apportées par EduMatch

Le projet EduMatch propose une solution innovante de mise en relation entre étudiants et professeurs en Tunisie, combinant les technologies modernes du web et l'intelligence artificielle pour répondre aux limites identifiées sur les plateformes existantes. La solution vise à faciliter l'accès à un enseignement personnalisé, efficace et transparent, tout en offrant une expérience utilisateur optimale.

EduMatch repose sur un système de matching intelligent, capable de calculer un score de compatibilité entre l'étudiant et le professeur en fonction de critères multiples : matière, niveau académique, disponibilité, localisation, budget et préférences pédagogiques. Cette approche permet de proposer les professeurs les mieux adaptés à chaque étudiant, réduisant ainsi le temps et les efforts nécessaires pour trouver un encadrement de qualité.

Un chatbot IA conversationnel, intégré à la plateforme, guide les étudiants dans leur recherche, répond à leurs questions et facilite la réservation de cours en ligne ou en présentiel. Cette fonctionnalité rend le parcours de l'utilisateur interactif, intuitif et rapide.

La plateforme comprend également un tableau de bord analytique complet pour les administrateurs, permettant de suivre les statistiques clés : nombre de réservations, matières les plus demandées,

professeurs les mieux notés, tendances par mois et analyse par domaine académique ou professionnel. Ces outils offrent une vision globale et détaillée des performances de la plateforme et permettent de prendre des décisions stratégiques basées sur les données.

Enfin, EduMatch intègre un système de gestion complet pour les professeurs et les administrateurs, incluant la validation des profils, la gestion des disponibilités, la communication avec les étudiants et le suivi des cours. Grâce à cette architecture, la plateforme combine personnalisation, sécurité, et efficacité, tout en se positionnant comme une alternative tunisienne robuste aux solutions internationales existantes.

1.7 Méthodologie de travail

Pour mener à bien notre projet EduMatch et assurer le bon déroulement de toutes les phases, notre équipe a adopté l'approche agile, en particulier Scrum comme cadre. Nous avons suivi ce cadre pendant toutes les phases de développement de notre projet. De plus, nous avons utilisé une approche inspirée de CRISP-DM pour guider nos processus liés à l'intégration de l'intelligence artificielle, notamment pour le chatbot et l'algorithme de matching.

1.7.1 Cadre de gestion de projet agile (Scrum)

Le modèle Agile est une approche moderne et bien définie pour développer de nouvelles idées en combinant des méthodes de gestion de projet établies. Basée sur un développement itératif et incrémental et menée avec une forte collaboration et communication d'équipe, cette méthodologie peut promouvoir une gestion et une organisation très efficaces pour gérer les nombreuses tâches d'un projet.

Scrum est un ensemble de processus utilisés pour gérer le travail sur des projets complexes depuis les années 1990. C'est davantage un cadre qui permet l'utilisation de différentes méthodes et techniques. Le cœur de Scrum est le "Sprint", qui est une période de deux à quatre semaines pendant laquelle un "incrément de produit" fonctionnel et potentiellement livrable est créé. Les

sprints ont une durée constante tout au long de la phase de développement. Un nouveau sprint commence immédiatement après la fin du sprint précédent.

1.7.2 Aperçu du projet

La figure 1.2 présente le cycle de vie de Scrum appliqué à notre projet EduMatch.

FIGURE 1.2 – Processus Scrum pour EduMatch

Pour mieux comprendre cette méthodologie, nous expliquerons les différents rôles des participants, les outils utilisés, et les différentes étapes d'un sprint Scrum.

a) A/ Acteurs principaux

La méthodologie Scrum implique trois rôles principaux :

- **Développeurs** : Ils représentent les collaborateurs engagés dans la création de l'incrément.

Les développeurs sont :

- Responsables de la création d'un incrément utilisable du produit à chaque sprint.
 - De la planification et de la création d'un sprint backlog.
 - Du maintien de la qualité en adhérant à la Définition de Fini (Definition of Done).
 - De l'adaptation quotidienne des plans pour atteindre l'objectif du sprint.
- **Product Owner** : Il peut être interne ou externe à l'entreprise. Son rôle fondamental est d'assurer la maximisation de la valeur du produit. Pour atteindre cet objectif, il :
 - Maximise la valeur du produit.
 - Développe et communique l'objectif du produit.
 - Crée et priorise les éléments du backlog produit.
 - Assure la transparence et la compréhension du backlog produit.
 - Prend des décisions concernant le backlog produit.
 - **Scrum Master** : Il est le leader et sa responsabilité principale est de s'assurer de l'établissement de Scrum suivant le guide et l'efficacité de l'équipe. Pour atteindre cet objectif, il :

- Établit et promeut les pratiques Scrum.
- Aide l'équipe à comprendre et à appliquer Scrum efficacement.
- Coach l'équipe sur l'auto-gestion et la création d'incréments de haute valeur.
- Facilite les événements Scrum et assure leur efficacité.
- Assiste le Product Owner dans la définition des objectifs et la gestion du backlog.
- Dirige et coach l'organisation dans l'adoption de Scrum.

Les membres de l'équipe Scrum pour le projet EduMatch sont présentés dans le tableau 1.4 :

TABLE 1.4 – Composition de l'équipe Scrum EduMatch

Équipe Scrum EduMatch	
Product Owner (PO)	[Nom du porteur du projet]
Scrum Master	[Nom de l'encadreur]
Équipe de développement	
Backend (FastAPI)	Mohamed Zouari
Frontend (React)	Mohamed Zouari
Base de données (PostgreSQL)	Mohamed Zouari
IA / Chatbot (LLM + RAG)	Mohamed Zouari
Intégration et Tests	Mohamed Zouari

b) B/ Événements Scrum

Les événements Scrum sont décrits dans le Guide Scrum pour créer de la régularité et minimiser le besoin d'autres réunions non planifiées. Pour notre projet EduMatch, ces événements sont :

- **Planification du Sprint** : Session collaborative où l'équipe Scrum définit le travail pour le Sprint, y compris la définition d'un objectif de Sprint et la sélection d'éléments du Backlog Produit. Pour EduMatch, chaque sprint planifiait les fonctionnalités à livrer (ex : Sprint 1 : authentification et profils, Sprint 2 : recherche et réservation, etc.).
- **Scrum quotidien (Daily Scrum)** : Brève réunion quotidienne permettant à l'équipe d'inspecter les progrès et de planifier le travail du lendemain. Ces réunions ont eu lieu régulièrement pour suivre l'avancement du backend, du frontend et de l'intégration IA.

- **Revue de Sprint** : Session pour examiner le résultat du Sprint, discuter des progrès vers l'objectif du produit, et adapter le Backlog Produit. À chaque fin de sprint, une démonstration des fonctionnalités développées a été réalisée.
- **Rétrospective de Sprint** : Réunion pour réfléchir au Sprint précédent et identifier les améliorations pour accroître la qualité et l'efficacité. Cela a permis d'ajuster la planification et d'optimiser le développement.

1.7.3 Planification des Sprints pour EduMatch

Le tableau 1.5 décrit la planification des sprints pour notre projet, où chaque sprint est détaillé avec ses tâches associées, conformément aux objectifs fonctionnels et techniques du projet.

TABLE 1.5 – Planification des sprints et tâches associées du projet

Sprint	Tâches
Sprint 0	— Analyse du cahier des charges et des besoins utilisateurs — Étude de la stack : FastAPI, React, PostgreSQL, Groq API — Mise en place de l'environnement de développement — Conception du modèle de données — Réflexion sur l'algorithme de matching
Sprint 1	— Implémentation du backend (FastAPI, SQLAlchemy) — Authentification JWT (inscription, connexion) — Développement frontend React (structure + routing) — Création des profils étudiant et professeur — Intégration du dark/light mode

TABLE 1.5 – Planification des sprints et tâches associées du projet

Sprint	Tâches
Sprint 2	— Module de recherche multi-filtres — Cartes professeurs (notes, tarifs) — Gestion des disponibilités — Système de réservation — Génération des liens Google Meet — Notifications temps réel
Sprint 3	— Développement du chatbot IA (EduBot) — Intégration LLM (Groq API - Llama 3.1) — Algorithme de matching multi-critères — Calcul du score de compatibilité — Extraction intelligente du niveau
Sprint 4	— Système d’avis et notation — Messagerie intégrée — Dashboard administrateur — Gestion Domaines/Niveaux/Matières — Validation des profils — Statistiques globales
Sprint 5	— Tests (unitaires, intégration, E2E) — Optimisation performances — Documentation (Swagger, README) — Préparation soutenance — Déploiement local

1.7.4 Cycle de vie de l'IA inspiré de CRISP-DM

Bien que notre projet EduMatch ne soit pas un projet de data mining pur, nous nous sommes inspirés du cadre CRISP-DM (Cross-Industry Standard Process for Data Mining) pour guider nos processus liés à l'intelligence artificielle, notamment pour le chatbot et l'algorithme de matching. Le cycle de vie CRISP-DM comprend six phases : Compréhension de l'entreprise, Compréhension des données, Préparation des données, Modélisation, Évaluation et Déploiement.

a) A/ Projets d'IA typiques vs projet EduMatch

Avant d'aborder l'application de CRISP-DM à EduMatch, comparons l'utilisation des données dans les projets d'IA traditionnels par rapport à notre projet.

Les projets d'IA traditionnels, comme la construction de modèles prédictifs, se concentrent sur l'analyse et l'interprétation des données existantes pour extraire des informations précieuses pour la prise de décision. Ils utilisent des méthodes structurées pour donner un sens à ces données. En revanche, notre projet EduMatch exploite l'IA de manière conversationnelle : le LLM est utilisé pour comprendre le langage naturel de l'étudiant, extraire ses besoins, et lui recommander des professeurs via une injection contextuelle (RAG) des données réelles de la base.

Contrairement aux projets d'IA générative qui créent de nouvelles données (images, texte), EduMatch utilise l'IA pour **interpréter, raisonner et recommander** à partir de données existantes structurées (profils professeurs, disponibilités, avis) et non structurées (bio, messages).

b) B/ Application de l'approche CRISP-DM à EduMatch

Appliquons les phases CRISP-DM à notre projet EduMatch :

- **Compréhension de l'entreprise** : Dans cette phase, nous avons analysé le contexte tunisien des cours particuliers, identifié les problèmes des étudiants (35% ont recours aux cours particuliers, recherche fragmentée) et des professeurs (gestion chaotique). Nous

avons défini les objectifs : plateforme intelligente avec chatbot IA, matching multi-critères, système de confiance.

- **Compréhension des données** : Nous avons identifié les données nécessaires au bon fonctionnement du système :
 - Données professeurs : matières, niveaux enseignés, tarifs, modes (en ligne/présentiel), disponibilités, notes, bio
 - Données étudiants : niveau scolaire, ville, budget, historique des réservations
 - Données référentielles : domaines, niveaux, matières, villes
- **Préparation des données** : Cette phase a consisté à :
 - Structurer la base de données PostgreSQL avec des relations claires
 - Nettoyer et normaliser les données de test (professeurs fictifs, disponibilités)
 - Préparer les embeddings ou contextes pour le RAG (injection des profils dans le prompt LLM)
 - Extraire intelligemment les niveaux enseignés depuis la bio des professeurs
- **Modélisation** : Nous avons développé plusieurs composants IA :
 - **Algorithme de matching multi-critères** : pondération sur 6 critères (matière, niveau, mode, budget, disponibilité, réputation) avec normalisation du score
 - **LLM (Llama 3.1 via Groq API)** : pour la compréhension du langage naturel et la génération des réponses conversationnelles
 - **RAG (Retrieval-Augmented Generation)** : injection des profils professeurs pertinents dans le contexte du LLM
 - **Extraction IA du niveau** : analyse de la bio du professeur pour déduire ses niveaux d'enseignement
- **Évaluation** : L'évaluation de notre système IA repose sur plusieurs métriques :
 - Précision de la compréhension des besoins étudiant par le chatbot
 - Pertinence des recommandations (top 3 professeurs)
 - Score de matching transparent et explicable (détail par critère)
 - Temps de réponse du chatbot (<5 secondes)

- Tests utilisateurs avec les personas définis
- **Déploiement** : Durant cette phase, le modèle et l'ensemble de l'application sont déployés :
 - Environnement local pour la démonstration (localhost)
 - Variables d'environnement pour les clés API (Groq)
 - Documentation Swagger auto-générée pour l'API
 - Scripts d'installation et README pour le déploiement local

1.7.5 Outils et bonnes pratiques

Pour assurer la qualité et la maintenabilité de notre projet, nous avons adopté les bonnes pratiques suivantes :

- **Gestion de version** : Git avec des commits réguliers et des messages explicites
- **Documentation** : Swagger auto-généré pour l'API.
- **Sécurité** : Hachage bcrypt des mots de passe, JWT pour l'authentification, validation Pydantic
- **Architecture modulaire** : Séparation claire des couches (routes, services, modèles) dans FastAPI; composants réutilisables dans React
- **Responsive design** : CSS Variables pour le theming dynamique (dark/light mode), adapté aux mobiles

1.8 Conclusion

Ce chapitre a présenté le cadre général de notre projet EduMatch. Après avoir introduit l'organisme d'accueil 21C'Digital et le contexte des cours particuliers en Tunisie, nous avons identifié les limites des solutions existantes et formulé la problématique.

Nous avons ensuite décrit la solution proposée ainsi que la méthodologie de travail adoptée basée sur Scrum et l'approche inspirée de CRISP-DM.

Le chapitre suivant détaillera l'analyse et la spécification des besoins du système.

Concepts Fondamentaux

Sommaire

2.1	INTRODUCTION	19
2.2	Intelligence Artificielle	19
2.2.1	Apprentissage Automatique (Machine Learning)	19
2.2.2	Deep Learning	20
2.3	Traitement Automatique du Langage Naturel (NLP)	21
2.3.1	Définition	21
2.3.2	Applications	21
2.3.3	Défis et Limites	22
2.4	Grands Modèles de Langage (LLM)	22
2.4.1	Définition	22
2.4.2	Caractéristiques des LLM	23
2.4.3	Principaux modèles LLM	23
2.4.4	Architecture des LLM	24
2.5	Optimisation des LLM pré-entraînés	25
2.5.1	Fine-tuning	25
2.5.2	RAG (Retrieval-Augmented Generation)	25
2.5.3	Prompt Engineering	26
2.5.4	Comparaison et choix pour EduMatch	26
2.6	Systèmes de Recommandation et Matching	27
2.6.1	Définition	27
2.6.2	Types de systèmes de recommandation	27
2.6.3	Algorithme de Matching d'EduMatch	28
2.7	Agents Conversationnels (Chatbots)	29
2.7.1	Définition et types	29
2.7.2	Architecture d'un chatbot à flux guidé hybride	29
2.8	Conclusion	30

2.1 INTRODUCTION

Ce chapitre présente les concepts fondamentaux qui sous-tendent le développement du système intelligent d'EduMatch, une plateforme de mise en relation entre apprenants et formateurs. Il couvre les notions clés de l'intelligence artificielle, de l'apprentissage automatique et du deep learning, avec un focus particulier sur les Grands Modèles de Langage (LLM). Le chapitre aborde également le rôle des LLM dans la compréhension du langage naturel, ainsi que leur utilisation dans les agents conversationnels, le prompt engineering et les systèmes de recommandation.

Dans le contexte d'EduMatch, ces technologies sont mobilisées pour concevoir **EduBot**, un assistant conversationnel intelligent capable d'extraire les besoins de l'apprenant en langage naturel (matière, niveau, budget, mode d'enseignement) et de recommander les formateurs les plus adaptés grâce à un algorithme de scoring multicritères.

2.2 Intelligence Artificielle

L'Intelligence Artificielle (IA) désigne l'ensemble des techniques et méthodes permettant à des systèmes informatiques d'effectuer des tâches qui requièrent normalement l'intelligence humaine : prise de décision, reconnaissance de formes, résolution de problèmes et compréhension du langage naturel [?].

Les applications de l'IA sont aujourd'hui omniprésentes : véhicules autonomes, reconnaissance vocale, assistants virtuels et diagnostic médical. Dans EduMatch, l'IA est exploitée pour interpréter les requêtes en langage naturel des apprenants et les transformer en critères de recherche exploitables par le moteur de matching.

2.2.1 Apprentissage Automatique (Machine Learning)

L'apprentissage automatique (ML) est une branche de l'IA qui se concentre sur la conception d'algorithmes permettant aux ordinateurs d'apprendre à partir de données et de prendre des

décisions sans être explicitement programmés [?]. Ces systèmes améliorent leurs performances au fil du temps grâce à l'exposition aux données.

Les principaux paradigmes d'apprentissage comprennent :

- **Apprentissage supervisé** : l'algorithme apprend à partir de données étiquetées. Il est utilisé pour des tâches de classification (ex : détection de spam) et de régression (ex : prédiction de prix). Dans EduMatch, ce paradigme pourrait servir à prédire la pertinence d'un formateur selon les préférences historiques d'un apprenant.
- **Apprentissage non supervisé** : l'algorithme découvre des structures cachées dans des données non étiquetées. Il est utile pour la segmentation de clientèle et la détection d'anomalies.
- **Apprentissage par renforcement** : un agent apprend par interaction avec son environnement en maximisant une récompense cumulée. Ce paradigme est largement utilisé en robotique et dans les systèmes de jeu.

2.2.2 Deep Learning

Le deep learning est une branche du ML qui exploite des réseaux de neurones artificiels à plusieurs couches pour apprendre des représentations complexes à partir des données [?]. Inspirés de la structure du cerveau humain, ces réseaux peuvent automatiquement extraire des caractéristiques pertinentes pour des tâches telles que la reconnaissance d'images, la synthèse vocale et le traitement du langage naturel.

Parmi les architectures les plus importantes :

- **Réseaux de Neurones Convolutifs (CNN)** : conçus pour le traitement des images, ils utilisent des couches de convolution pour extraire des caractéristiques visuelles (contours, textures, formes).
- **Réseaux de Neurones Récurrents (RNN)** : adaptés aux données séquentielles, ils maintiennent une mémoire interne pour contextualiser les entrées dans le temps. Ils sont utilisés en traitement du langage naturel et en reconnaissance vocale.

- **Transformers** : architecture de deep learning introduite par [?], basée sur un mécanisme d'auto-attention. Contrairement aux RNN, les Transformers traitent toutes les positions d'une séquence en parallèle, ce qui leur permet de capturer des dépendances à longue portée et d'être entraînés plus efficacement. Cette architecture constitue le fondement de tous les LLM modernes.

2.3 Traitement Automatique du Langage Naturel (NLP)

2.3.1 Définition

Le Traitement Automatique du Langage Naturel (NLP, *Natural Language Processing*) est une branche de l'IA qui se concentre sur la capacité des ordinateurs à comprendre, interpréter, générer et répondre au langage humain de manière significative et contextuelle [?]. Le NLP constitue le pont entre la communication humaine et la compréhension informatique.

2.3.2 Applications

Le NLP couvre un large spectre d'applications :

- **Chatbots et assistants virtuels** : automatisent les interactions avec les utilisateurs, comme EduBot dans EduMatch, qui guide l'apprenant pour définir son besoin de formation.
- **Traduction automatique** : des outils comme Google Translate convertissent des textes en préservant les nuances linguistiques et contextuelles.
- **Analyse de sentiments** : analyse les avis des utilisateurs pour évaluer leurs opinions sur des produits ou services.
- **Moteurs de recherche** : améliorent la compréhension du contexte des mots pour fournir des résultats plus pertinents.
- **Extraction d'informations** : identifie et structure automatiquement les entités et relations dans un texte brut.

2.3.3 Défis et Limites

Malgré ses progrès remarquables, le NLP fait face à plusieurs défis :

- **Compréhension contextuelle** : saisir les subtilités du langage humain (sarcasme, idiomes, ambiguïté) reste difficile et peut mener à des malinterprétations.
- **Variation linguistique** : la diversité des langues et des dialectes complique la conception de systèmes universels.
- **Ambiguïté** : un même mot ou une même phrase peut avoir des significations différentes selon le contexte, induisant des erreurs dans les systèmes NLP.
- **Qualité et quantité des données** : les performances des systèmes NLP dépendent fortement de la qualité et de la représentativité des données d'entraînement, qui peuvent contenir des biais.

L'émergence des Grands Modèles de Langage (LLM) apporte des solutions plus avancées à ces défis, en permettant une meilleure intégration du contexte et une compréhension plus fine du langage naturel.

2.4 Grands Modèles de Langage (LLM)

2.4.1 Définition

Les Grands Modèles de Langage (LLM, *Large Language Models*) représentent une avancée majeure dans le domaine du NLP [?]. Ce sont des modèles d'apprentissage profond, basés principalement sur l'architecture Transformer, capables de comprendre, générer et manipuler le langage humain à une échelle et une sophistication sans précédent. Entraînés sur des corpus textuels massifs et diversifiés, les LLM apprennent des patterns linguistiques complexes, des relations contextuelles et des nuances sémantiques.

Le tableau 2.1 présente une comparaison entre les approches NLP traditionnelles et les LLM.

TABLE 2.1 – Comparaison entre NLP traditionnel et LLM

Caractéristique	NLP Traditionnel	LLM
Périmètre	Méthodes spécialisées par tâche (classification, traduction...)	Modèle généraliste gérant de multiples tâches
Architecture	Techniques classiques (tokenisation, analyse syntaxique, NER)	Transformers avec mécanismes d'attention
Données d'entraînement	Corpus spécialisés et annotés	Données massives hétérogènes (web, livres, forums)
Performance	Dépend des techniques et du prétraitement	Robuste et polyvalent grâce au volume de données

2.4.2 Caractéristiques des LLM

Les LLM se distinguent par plusieurs caractéristiques clés [?] :

- **Entraînement sur des corpus massifs** : ces modèles sont entraînés sur des corpus textuels immenses issus de sources variées, leur permettant de générer du contenu diversifié dans de multiples registres linguistiques.
- **Polyvalence** : grâce à leur entraînement généraliste, les LLM peuvent effectuer une grande variété de tâches sans nécessiter de ré-entraînement spécifique.
- **Compréhension contextuelle** : ils maintiennent la cohérence textuelle et produisent des réponses adaptées au contexte sur de longues conversations.
- **Évolution continue** : exposés régulièrement à de nouvelles données, les LLM peuvent enrichir leurs connaissances et intégrer les évolutions du langage.

2.4.3 Principaux modèles LLM

Plusieurs LLM ont eu un impact majeur sur la recherche et les applications industrielles :

- **GPT (Generative Pre-trained Transformer) [?]** : développé par OpenAI, GPT-3.5 et GPT-4 ont établi des records dans la génération de texte, les systèmes de dialogue et la génération de code.
- **BERT (Bidirectional Encoder Representations from Transformers) [?]** : introduit par Google, BERT traite le texte dans les deux directions, capturant des relations sémantiques

profondes. Il est largement utilisé pour la classification et la reconnaissance d'entités nommées.

- **LLaMA (Large Language Model Meta AI)** [?] : développé par Meta AI, LLaMA offre de hautes performances avec une empreinte computationnelle réduite. Son caractère open-source en fait un choix privilégié pour la recherche et le déploiement applicatif.
- **PaLM (Pathways Language Model)** [?] : développé par Google, PaLM démontre de fortes capacités de raisonnement, de traduction et de question-réponse.

Dans le cadre d'EduMatch, nous utilisons le modèle **LLaMA 3.1** via l'API **Groq**, qui offre des temps d'inférence très faibles (latence < 500ms), essentiel pour une expérience conversationnelle fluide dans EduBot.

2.4.4 Architecture des LLM

L'architecture des LLM repose fondamentalement sur le modèle Transformer introduit par Vaswani et al. en 2017 [?]. Ses composants essentiels sont :

- **Couche d'embedding** : convertit chaque token (mot ou sous-mot) en un vecteur dense préservant l'information sémantique.
- **Encodage positionnel** : ajoute une information sur la position des tokens dans la séquence, compensant l'absence de séquentialité dans les Transformers.
- **Mécanisme d'auto-attention multi-têtes** : permet au modèle de pondérer l'importance de différents mots les uns par rapport aux autres, capturant divers types de relations en parallèle.
- **Réseaux de neurones feed-forward** : appliquent des transformations non-linéaires à chaque sortie d'attention pour enrichir les représentations.
- **Normalisation et connexions résiduelles** : stabilisent et accélèrent l'entraînement en préservant le flux de gradient.
- **Couche de sortie** : une couche softmax prédit la distribution de probabilité du prochain token, permettant la génération de texte.

Les LLM modernes contiennent des milliards de paramètres, certains dépassant 100 milliards, ce qui leur permet de capturer des patterns linguistiques complexes et des dépendances à longue portée.

2.5 Optimisation des LLM pré-entraînés

Bien que les LLM soient remarquablement performants sur des tâches générales, leur adaptation à des contextes spécialisés nécessite des techniques d'optimisation. Trois approches principales existent.

2.5.1 Fine-tuning

Le fine-tuning consiste à réentraîner un LLM pré-entraîné sur un ensemble de données spécifique à un domaine pour adapter ses performances [?]. Le modèle apprend les nuances, la terminologie et les subtilités contextuelles du domaine ciblé.

Il se distingue du pré-entraînement initial par :

- **Pré-entraînement** : part de paramètres initialisés aléatoirement et s'entraîne sur de vastes corpus non étiquetés par apprentissage auto-supervisé.
- **Fine-tuning** : part des poids du modèle pré-entraîné comme point de départ et ré-entraîne sur un dataset plus petit et plus ciblé.

2.5.2 RAG (Retrieval-Augmented Generation)

La Génération Augmentée par Récupération (RAG) est une technique avancée qui enrichit la génération de texte en incorporant des connaissances externes récupérées depuis des sources de données privées ou propriétaires [?]. Le pipeline RAG fonctionne en cinq étapes :

1. **Encodage de la requête** : la requête utilisateur est encodée en un vecteur dense représentant sa sémantique.

2. **Récupération de documents** : une recherche de similarité est effectuée dans une base de documents pré-indexée pour extraire les passages les plus pertinents.
3. **Fusion contextuelle** : les documents récupérés sont concaténés avec la requête originale pour former une entrée enrichie.
4. **Génération de texte** : le modèle génératif produit une réponse cohérente en s'appuyant sur la requête et le contexte récupéré.
5. **Post-traitement (optionnel)** : amélioration de la lisibilité et filtrage des hallucinations.

2.5.3 Prompt Engineering

Le prompt engineering désigne la méthode systématique de conception d'instructions pour guider un LLM vers des réponses plus précises, pertinentes et contextuellement appropriées [?].

Un prompt bien structuré comprend généralement :

- **Instruction** : indique au modèle la tâche à effectuer et le format de sortie attendu.
- **Contexte externe** : informations supplémentaires pour enrichir la réponse du modèle.
- **Requête utilisateur** : la demande réelle de l'utilisateur.
- **Indicateur de sortie** (optionnel) : signal indiquant où le modèle doit commencer sa génération.

2.5.4 Comparaison et choix pour EduMatch

Le tableau 2.2 compare les trois approches d'optimisation.

Choix pour EduMatch : Dans notre projet, le **prompt engineering** est privilégié par rapport au RAG et au fine-tuning pour les raisons suivantes :

- La base de connaissances d'EduMatch (formateurs, niveaux, matières) est déjà structurée dans une base de données relationnelle PostgreSQL. Le RAG ajouterait une complexité inutile puisque l'extraction des critères de la conversation suffit pour interroger directement la base.

TABLE 2.2 – Comparaison Fine-tuning, RAG et Prompt Engineering

Aspect	Fine-tuning	RAG	Prompt Engineering
Objectif	Adapter le modèle à un domaine spécifique	Combiner récupération d'info et génération	Guider le modèle via des instructions
Modification du modèle	Oui (poids modifiés)	Non	Non
Coût	Élevé (GPU, données étiquetées)	Modéré (infrastructure de récupération)	Faible
Latence	Faible à l'inférence	Variable (dépend de la récupération)	Faible
Adaptabilité	Faible (re-training nécessaire)	Élevée (base de docs mise à jour)	Très élevée

- Le prompt engineering offre un contrôle précis sur l'extraction des critères (matière, niveau, budget, mode, ville) et la génération de réponses naturelles, tout en garantissant une faible latence.
- Cette approche s'intègre naturellement avec l'API Groq (LLaMA 3.1-8b-instant), offrant des inférences rapides sans infrastructure additionnelle.

2.6 Systèmes de Recommandation et Matching

2.6.1 Définition

Un système de recommandation est un outil qui prédit les préférences d'un utilisateur pour lui suggérer des items pertinents [?]. Ces systèmes sont omniprésents dans les plateformes numériques modernes : Netflix pour les films, Amazon pour les produits, LinkedIn pour les offres d'emploi.

2.6.2 Types de systèmes de recommandation

- **Filtrage collaboratif** : recommande des items en se basant sur les préférences d'utilisateurs similaires (ex : "les utilisateurs qui ont aimé X ont aussi aimé Y").

- **Filtrage basé sur le contenu** : recommande des items similaires à ceux qu'un utilisateur a appréciés par le passé, en analysant les caractéristiques des items.
- **Approche hybride** : combine les deux méthodes pour améliorer la précision des recommandations.
- **Scoring multicritères** : calcule un score de pertinence en pondérant plusieurs critères explicites fournis par l'utilisateur.

2.6.3 Algorithme de Matching d'EduMatch

Dans EduMatch, nous avons opté pour un **système de scoring multicritères** adapté au contexte éducatif tunisien. L'algorithme calcule un score de compatibilité sur 100 points pour chaque formateur, basé sur six dimensions pondérées :

TABLE 2.3 – Pondération des critères de matching EduMatch

Critère	Poids	Justification
Matière	25 pts	Correspondance entre la matière demandée et les matières enseignées
Niveau	40 pts	Critère le plus discriminant — un formateur Lycée ne convient pas au Primaire
Ville	15 pts	Proximité géographique (présentiel uniquement)
Budget	10 pts	Compatibilité entre tarif du formateur et budget de l'apprenant
Mode	5 pts	Correspondance en ligne / présentiel / les deux
Réputation	5 pts	Note moyenne des avis étudiants
Total	100 pts	

Les formateurs sont ensuite classés par score décroissant et les 3 meilleurs profils sont présentés à l'apprenant avec un label de qualité :

- **Excellent** : score $\geq 85/100$
- **Très bon** : score $\in [70, 84]$
- **Bon** : score $\in [55, 69]$
- **Possible** : score < 55

2.7 Agents Conversationnels (Chatbots)

2.7.1 Définition et types

Un agent conversationnel (chatbot) est un programme informatique conçu pour simuler une conversation humaine via des interfaces textuelles ou vocales [?]. On distingue deux grandes familles :

- **Chatbots à règles** : basés sur des arbres de décision et des patterns prédéfinis. Ils sont prévisibles mais limités à des scénarios rigides.
- **Chatbots basés sur les LLM** : exploitent des modèles de langage pré-entraînés pour comprendre et générer du texte en langage naturel. Ils sont plus flexibles et capables de gérer des conversations ouvertes.

2.7.2 Architecture d'un chatbot à flux guidé hybride

EduBot adopte une architecture **hybride** qui combine :

- Un **flux guidé structuré** pour la collecte séquentielle des critères (domaine → matière → niveau → budget → mode → ville), garantissant la complétude des informations nécessaires au matching.
- Un **LLM (LLaMA 3.1 via Groq)** pour deux tâches distinctes :
 1. **Extraction NLP** : analyse toute la conversation et extrait les critères structurés en JSON (matière, niveau, budget, mode, ville).
 2. **Génération naturelle** : produit des réponses conversationnelles fluides et contextualisées (confirmations, questions de suivi, annonce des résultats).

Cette architecture présente plusieurs avantages :

- **Robustesse** : le flux guidé garantit que tous les critères nécessaires au matching sont collectés avant d'afficher les résultats.
- **Naturalité** : le LLM assure des échanges fluides, évitant les réponses robotiques et rigides des chatbots à règles.

- **Faible latence** : l'API Groq offre des inférences en moins de 500ms, assurant une expérience utilisateur réactive.
- **Gestion des alternatives** : si aucun formateur n'est trouvé pour une matière donnée, EduBot propose automatiquement des matières alternatives pertinentes.

2.8 Conclusion

Dans ce chapitre, nous avons exploré les concepts fondamentaux qui constituent le socle technologique d'EduMatch. Nous avons notamment couvert l'intelligence artificielle, le machine learning, le deep learning et l'architecture Transformer, qui constituent la base des LLM modernes.

Nous avons ensuite présenté les techniques d'optimisation des LLM — fine-tuning, RAG et prompt engineering — et justifié le choix du prompt engineering pour EduBot, qui s'avère le mieux adapté à notre contexte : une base de données relationnelle structurée, une contrainte de faible latence et un besoin de contrôle précis sur l'extraction des critères.

Enfin, nous avons présenté l'algorithme de scoring multicritères qui constitue le cœur du système de matching d'EduMatch, attribuant à chaque formateur un score de compatibilité pondéré selon six dimensions (matière, niveau, ville, budget, mode, réputation) et permettant de recommander les trois profils les plus adaptés à chaque apprenant.

Le chapitre suivant détaillera l'analyse des besoins et la conception de la plateforme EduMatch.

Concepts Fondamentaux

Sommaire

3.1	Introduction	32
3.2	Intelligence Artificielle et NLP	32
3.3	Modèles de Langage (LLM)	32
3.4	Prompt Engineering	33
3.5	Agents Conversationnels (Chatbot)	33
3.6	Système de Recommandation et Matching	34
3.7	Conclusion	34

3.1 Introduction

Ce chapitre présente les concepts fondamentaux utilisés dans le développement de la plateforme **EduMatch**, notamment les notions d'intelligence artificielle, de traitement du langage naturel (NLP), de modèles de langage (LLM), ainsi que les systèmes de recommandation et les agents conversationnels.

Ces technologies sont exploitées pour concevoir **EduBot**, un chatbot intelligent capable de comprendre les besoins des apprenants et de recommander les formateurs les plus adaptés grâce à un système de matching.

3.2 Intelligence Artificielle et NLP

L'intelligence artificielle (IA) regroupe les techniques permettant à une machine de simuler des capacités humaines telles que la compréhension, l'apprentissage et la prise de décision.

Dans le cadre d'EduMatch, l'IA est principalement utilisée à travers le **Traitement Automatique du Langage Naturel (NLP)**, qui permet d'analyser les requêtes textuelles des utilisateurs.

Le NLP permet notamment :

- d'extraire les informations clés (matière, niveau, budget, ville)
- de comprendre les intentions de l'utilisateur
- de générer des réponses naturelles

3.3 Modèles de Langage (LLM)

Les **Grands Modèles de Langage (LLM)** sont des modèles d'intelligence artificielle capables de comprendre et générer du texte en langage naturel.

Ils sont basés sur l'architecture Transformer et sont entraînés sur de grandes quantités de données textuelles, ce qui leur permet de produire des réponses cohérentes et contextualisées.

Dans notre projet, nous utilisons le modèle **LLaMA 3.1** via l'API **Groq**, permettant :

- une compréhension avancée des requêtes utilisateurs
- une génération de réponses naturelles
- une faible latence adaptée aux systèmes interactifs

3.4 Prompt Engineering

Le **prompt engineering** consiste à concevoir des instructions précises afin de guider le modèle de langage vers des réponses pertinentes.

Dans EduMatch, cette technique est utilisée pour :

- extraire automatiquement les critères de recherche (matière, niveau, budget, mode, ville)
- structurer les informations sous forme exploitable
- améliorer la qualité des réponses du chatbot

Cette approche a été privilégiée car elle est simple, efficace et ne nécessite pas d'entraînement supplémentaire du modèle.

3.5 Agents Conversationnels (Chatbot)

Un chatbot est un système capable de simuler une conversation avec un utilisateur.

Dans EduMatch, **EduBot** repose sur une architecture hybride combinant :

- un **flux guidé** pour collecter les informations nécessaires
- un **LLM** pour comprendre et générer des réponses naturelles

Ce chatbot permet :

- de guider l'utilisateur dans sa recherche
- de simplifier la saisie des critères
- d'améliorer l'expérience utilisateur

3.6 Système de Recommandation et Matching

Un système de recommandation permet de proposer des résultats adaptés aux besoins d'un utilisateur.

Dans EduMatch, nous avons implémenté un **système de scoring multicritères** qui attribue un score de compatibilité à chaque formateur selon plusieurs critères :

- matière
- niveau
- localisation
- budget
- mode (en ligne / présentiel)
- réputation

Les formateurs sont ensuite classés selon leur score, et les meilleurs profils sont proposés à l'apprenant.

Cette approche permet d'optimiser la pertinence des recommandations et de réduire le temps de recherche.

3.7 Conclusion

Dans ce chapitre, nous avons les concepts fondamentaux utilisés dans le développement de la plateforme EduMatch. Nous avons mis en évidence le rôle de l'intelligence artificielle, du NLP et des modèles de langage dans la conception du chatbot EduBot.

Nous avons également présenté le système de recommandation basé sur un algorithme de matching, qui constitue le cœur de la plateforme.

Le chapitre suivant sera consacré à l'analyse et à la spécification des besoins du système.

Analyse, Conception et Architecture du Système

Sommaire

4.1	Introduction	36
4.2	Étude des besoins	36
4.2.1	Identification des acteurs	36
4.2.2	Besoins fonctionnels	37
4.2.3	Besoins non fonctionnels	38
4.3	Conception UML	39
4.3.1	Présentation de l'UML	39
4.3.2	Diagramme des cas d'utilisation	39
4.3.3	Diagramme d'activité	42
4.3.4	Diagramme de séquence	44
4.3.5	Diagramme de classes	45
4.4	Architecture Technique	47
4.4.1	Architecture 3-tiers	47
4.4.2	Architecture du chatbot EduBot	49
4.4.3	Workflow de validation des formateurs	50
4.5	Conclusion	4

4.1 Introduction

La conception d'un système informatique constitue une étape fondamentale dans le processus de développement logiciel. Elle permet de définir la structure, les comportements et les interactions entre les composants avant toute implémentation.

Ce chapitre présente l'analyse et la conception de la plateforme. Nous commençons par identifier les acteurs et analyser les besoins fonctionnels et non fonctionnels du système. Nous exposons ensuite les diagrammes UML (cas d'utilisation, activité, séquence et classes) avant de détailler l'architecture technique et le pipeline du chatbot intelligent.

4.2 Étude des besoins

Avant de passer à la conception UML, il est essentiel d'analyser les besoins du système afin de garantir que la plateforme réponde correctement aux attentes des différents utilisateurs.

4.2.1 Identification des acteurs

Les acteurs sont des entités extérieures au système qui interagissent directement avec lui, soit pour exploiter ses services, soit pour lui en fournir. Dans EduMatch, nous distinguons trois acteurs principaux :

- **Étudiant / Apprenant** : acteur principal côté demande. Il utilise la plateforme pour rechercher un formateur adapté à ses besoins, interagir avec le chatbot EduBot, consulter des profils, réserver des séances et laisser des avis après les cours.
- **Formateur / Professeur** : acteur principal côté offre. Il crée et complète son profil (bio, tarifs, matières enseignées, disponibilités), soumet son dossier pour validation, gère ses réservations et répond aux messages des apprenants.
- **Administrateur** : acteur de supervision. Il valide les profils des formateurs, gère le référentiel académique et professionnel (domaines, niveaux, matières), traite les demandes de nouvelles matières et consulte les analytiques de la plateforme.

4.2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités attendues du système telles que perçues par ses utilisateurs. Pour EduMatch, ces besoins sont les suivants :

- **Authentification et gestion des comptes** : Le système doit permettre l'inscription, la connexion et la déconnexion sécurisées pour les trois types d'acteurs. L'authentification repose sur un mécanisme de token JWT. L'accès aux fonctionnalités est contrôlé selon le rôle de l'utilisateur (étudiant, formateur, administrateur).
- **Recherche et filtrage de formateurs** : L'apprenant doit pouvoir rechercher des formateurs selon plusieurs critères combinables : matière, niveau, ville, budget, mode d'enseignement (en ligne / présentiel) et domaine (académique / professionnel). Les résultats sont triés par note moyenne et peuvent être sauvegardés en favoris.
- **Chatbot intelligent EduBot** : Le système doit proposer un assistant conversationnel capable de comprendre les besoins de l'apprenant en langage naturel, de collecter ses critères de recherche de manière progressive (matière, niveau, budget, mode, ville) et de recommander les trois formateurs les plus adaptés avec un score de matching détaillé. EduBot doit gérer les sessions de conversation persistantes et proposer des alternatives si aucun résultat exact n'est trouvé.
- **Gestion du profil formateur** : Le formateur doit pouvoir renseigner et mettre à jour ses informations personnelles (bio, ville, téléphone), ses tarifs (en ligne et présentiel), son mode d'enseignement, ses matières enseignées avec niveaux associés, ainsi que ses documents justificatifs (photo, CV, diplômes, certificats). Un indicateur de progression guide le formateur vers la complétion de son profil.
- **Workflow de validation des formateurs** : Le système doit implémenter un processus de validation en quatre étapes : inscription (statut *incomplet*), complétion du profil, soumission à l'administrateur (statut *en_attente*), puis validation ou refus (statut *validé* ou *refusé*). Un profil incomplet ne peut pas être soumis à validation.
- **Gestion des disponibilités et réservations** : Le formateur doit pouvoir créer et gérer ses créneaux horaires disponibles (date, heure, mode, capacité). L'apprenant doit pouvoir

réserver un créneau disponible auprès d'un formateur validé. Le système doit gérer les statuts de réservation (en attente, confirmée, annulée) et contrôler les places disponibles.

- **Système d'avis et de notation** : Après une séance, l'apprenant doit pouvoir laisser un avis (note de 1 à 5 et commentaire) sur le formateur. La note moyenne du formateur est recalculée automatiquement et influence son score de matching dans EduBot.
- **Messagerie directe** : Le système doit permettre une messagerie directe entre un apprenant et un formateur, indépendante du chatbot EduBot. Les messages non lus doivent être signalés via le système de notifications.
- **Gestion du référentiel** : L'administrateur doit pouvoir gérer la hiérarchie de référentiel : Domaine → Niveau → Matière pour les deux filières (académique et professionnelle). Il doit également traiter les demandes de nouvelles matières proposées par les formateurs.
- **Tableau de bord et analytiques** : L'administrateur doit disposer d'un tableau de bord affichant les statistiques clés de la plateforme : nombre d'utilisateurs, formateurs validés, réservations, revenus et demandes en attente. Des notifications en temps réel l'alertent des actions requises (nouveaux profils à valider, demandes de matières).

4.2.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité et les critères de performance que le système doit respecter.

- **Performance** : Le système doit garantir des temps de réponse rapides pour les opérations principales telles que la recherche, les réservations et les interactions avec le chatbot.
- **Sécurité** : L'accès au système doit être sécurisé par authentification. Les mots de passe doivent être stockés de manière sécurisée (hachage). L'accès aux fonctionnalités doit être contrôlé selon les rôles.
- **Disponibilité** : La plateforme doit être accessible en continu avec un minimum d'interruptions.
- **Ergonomie** : L'interface doit être simple, intuitive et adaptée à différents supports (ordinateur, tablette, mobile).

- **Scalabilité et maintenabilité** : L'architecture du système doit permettre son évolution et faciliter la maintenance ainsi que l'ajout de nouvelles fonctionnalités.
- **Extensibilité** : Le système de recommandation doit être conçu de manière flexible afin de permettre l'ajustement des critères de matching sans modifier l'ensemble du système.

4.3 Conception UML

4.3.1 Présentation de l'UML

L'UML (*Unified Modeling Language*) est un langage de modélisation orienté objet qui offre une approche visuelle pour concevoir et représenter un système de manière standardisée [?]. Son utilisation présente plusieurs avantages pour un projet comme EduMatch :

- Définir une architecture robuste et cohérente avant l'implémentation
- Modéliser les interactions entre les acteurs et le système
- Faciliter la communication entre les membres de l'équipe
- Réduire les ambiguïtés grâce à une notation formelle standardisée
- Identifier et traiter les points critiques en amont du développement

Nous utilisons dans ce chapitre trois types de diagrammes UML complémentaires : le diagramme de cas d'utilisation, le diagramme de classe et le diagramme de séquence.

4.3.2 Diagramme des cas d'utilisation

4.3.2.1 Définition

Le diagramme de cas d'utilisation joue un rôle central dans la modélisation UML en décrivant les interactions entre les acteurs et les fonctionnalités du système. Il constitue le point d'articulation entre l'utilisateur et les différentes fonctionnalités mises en œuvre. Son objectif est de rassembler, analyser et organiser les besoins des utilisateurs en les exprimant sous forme de cas d'utilisation cohérents.

4.3.2.2 Composition du diagramme

Un diagramme de cas d'utilisation comprend :

- **Un acteur** : entité extérieure qui interagit avec le système (une personne, un rôle ou un système externe).
- **Un cas d'utilisation** : unité cohérente représentant une fonctionnalité visible de l'extérieur, réalisant un service complet pour l'acteur qui l'initie.
- **Les relations** : association simple (acteur $\leftrightarrow$ cas d'utilisation), relation d'inclusion («*include*»), relation d'extension («*extend*») et héritage entre acteurs.

4.3.2.3 Diagramme global d'EduMatch

La figure 4.1 présente le diagramme de cas d'utilisation global d'EduMatch avec ses trois acteurs et leurs fonctionnalités associées.

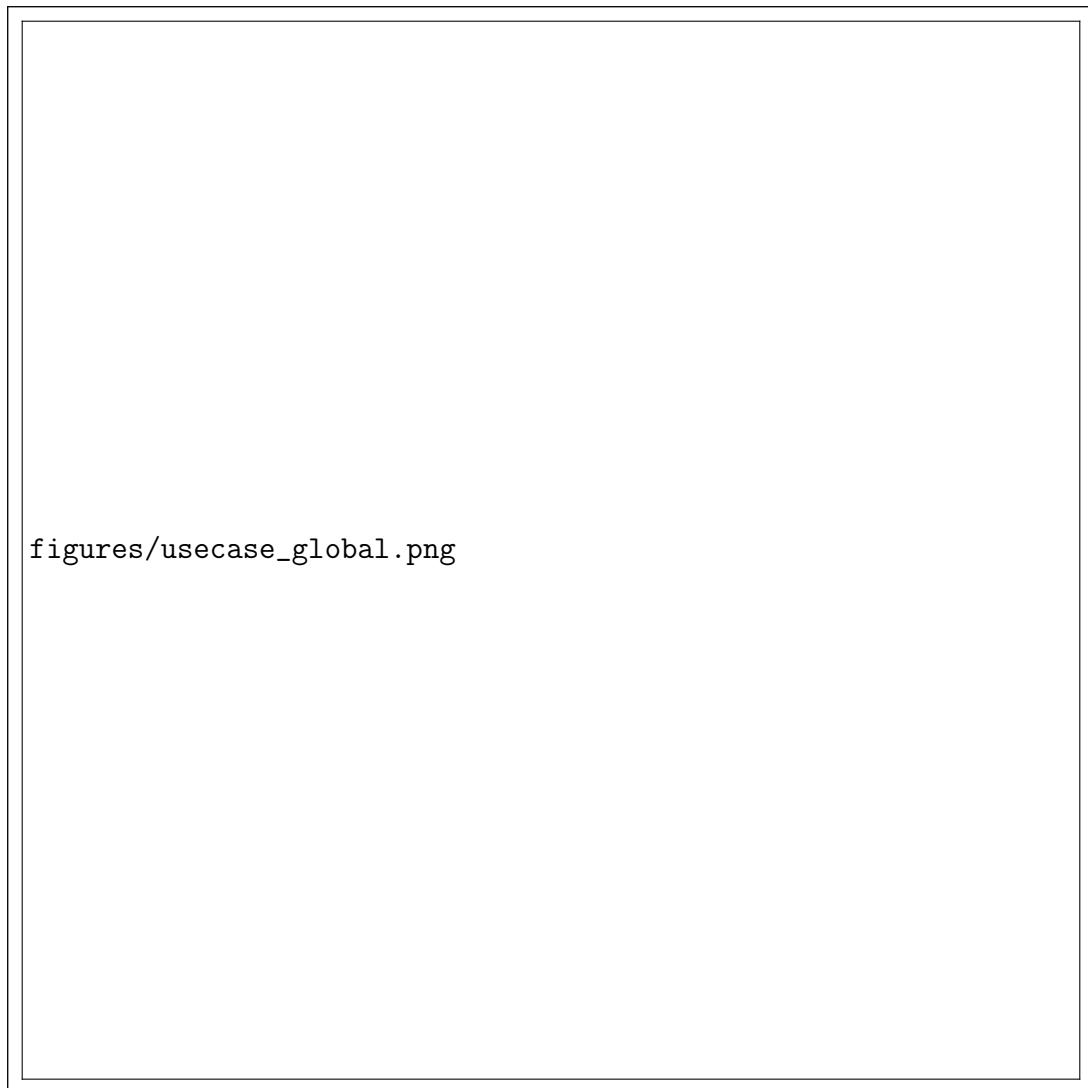


FIGURE 4.1 – Diagramme de cas d'utilisation global — EduMatch

Le tableau 4.1 fournit une description concise des principaux cas d'utilisation.

TABLE 4.1 – Description des cas d'utilisation principaux

Cas d'utilisation	Description
S'inscrire / Se connecter	Créer un compte ou accéder à la plateforme avec authentification JWT
Rechercher un formateur	Filtrer les formateurs validés par matière, niveau, ville, budget et mode
Interagir avec EduBot	Dialoguer avec le chatbot IA pour obtenir des recommandations personnalisées
Réserver une séance	Sélectionner un créneau disponible et effectuer une réservation
Laisser un avis	Évaluer un formateur après une séance (note 1-5 + commentaire)
Gérer son profil	Compléter et mettre à jour ses informations, tarifs, matières et documents
Soumettre pour validation	Envoyer le profil complété à l'administrateur pour approbation
Valider un formateur	Examiner et approuver ou refuser un profil de formateur soumis
Gérer le référentiel	Administrer la hiérarchie Domaine → Niveau → Matière
Consulter les analytiques	Visualiser les statistiques et indicateurs clés de la plateforme

4.3.3 Diagramme d'activité

4.3.3.1 Définition

Le diagramme d'activité est un diagramme comportemental UML qui modélise le flux d'activités au sein d'un système. Il décrit la séquence d'actions, les points de décision

et les flux parallèles d'un processus, permettant de visualiser la logique interne d'un cas d'utilisation.

4.3.3.2 Flux de recherche via EduBot

La figure 4.2 illustre le flux d'activité du chatbot EduBot lors d'une session de recherche de formateur.

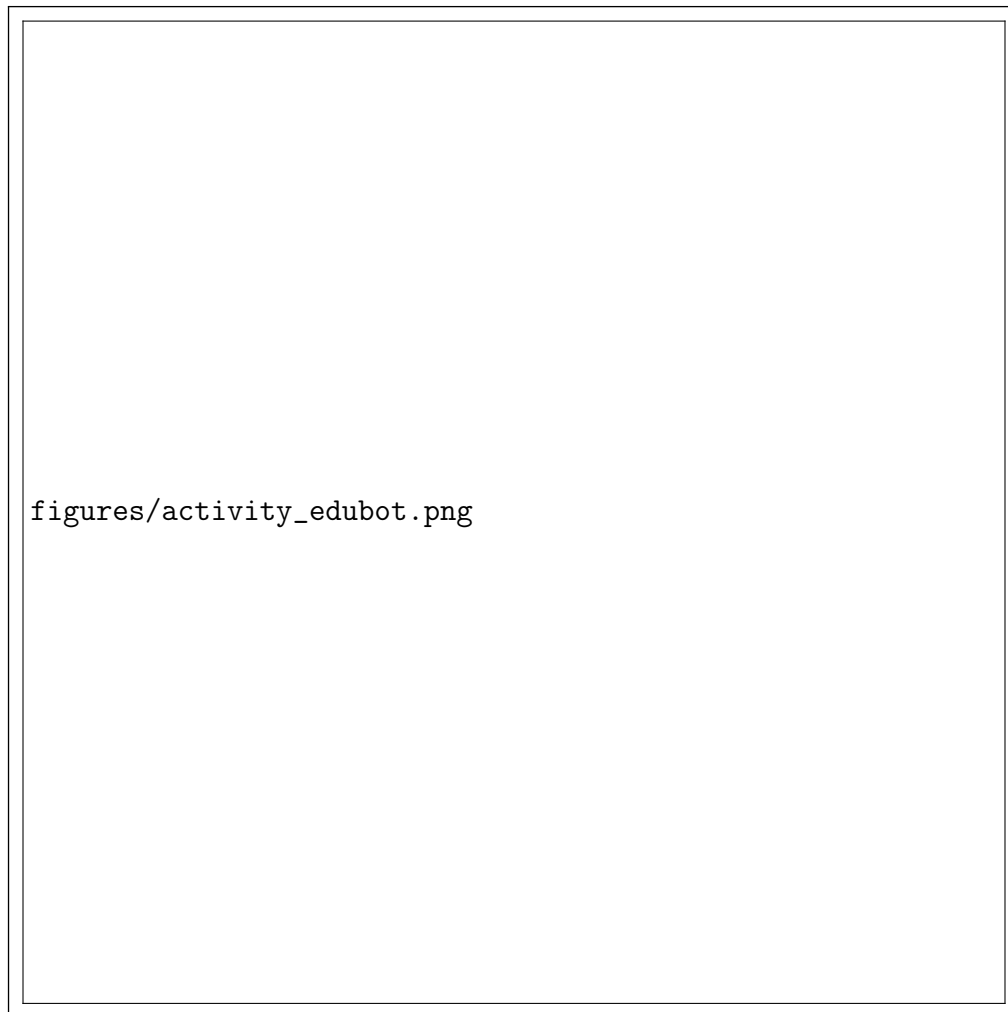


FIGURE 4.2 – Diagramme d'activité — Flux de recherche via EduBot

Les principales étapes du flux sont :

1. L'apprenant envoie un message en langage naturel.
2. EduBot extrait les critères présents via le LLM (LLaMA 3.1 / Groq API).
3. Si des critères manquent, EduBot pose une question ciblée.

4. Une fois tous les critères collectés, le moteur de scoring calcule le score de chaque formateur validé sur 100 points.
5. Les trois meilleurs profils sont retournés avec scores, créneaux disponibles et avertissements.
6. Si aucun résultat n'est trouvé, EduBot propose des matières alternatives.

4.3.4 Diagramme de séquence

4.3.4.1 Définition

Le diagramme de séquence est un diagramme d'interaction UML qui modélise les échanges de messages entre les objets du système dans un ordre temporel précis. Il est particulièrement utile pour décrire le comportement dynamique d'un cas d'utilisation complexe.

4.3.4.2 Processus de réservation d'une séance

La figure 4.3 présente les interactions entre les composants lors d'une réservation de séance par un apprenant.

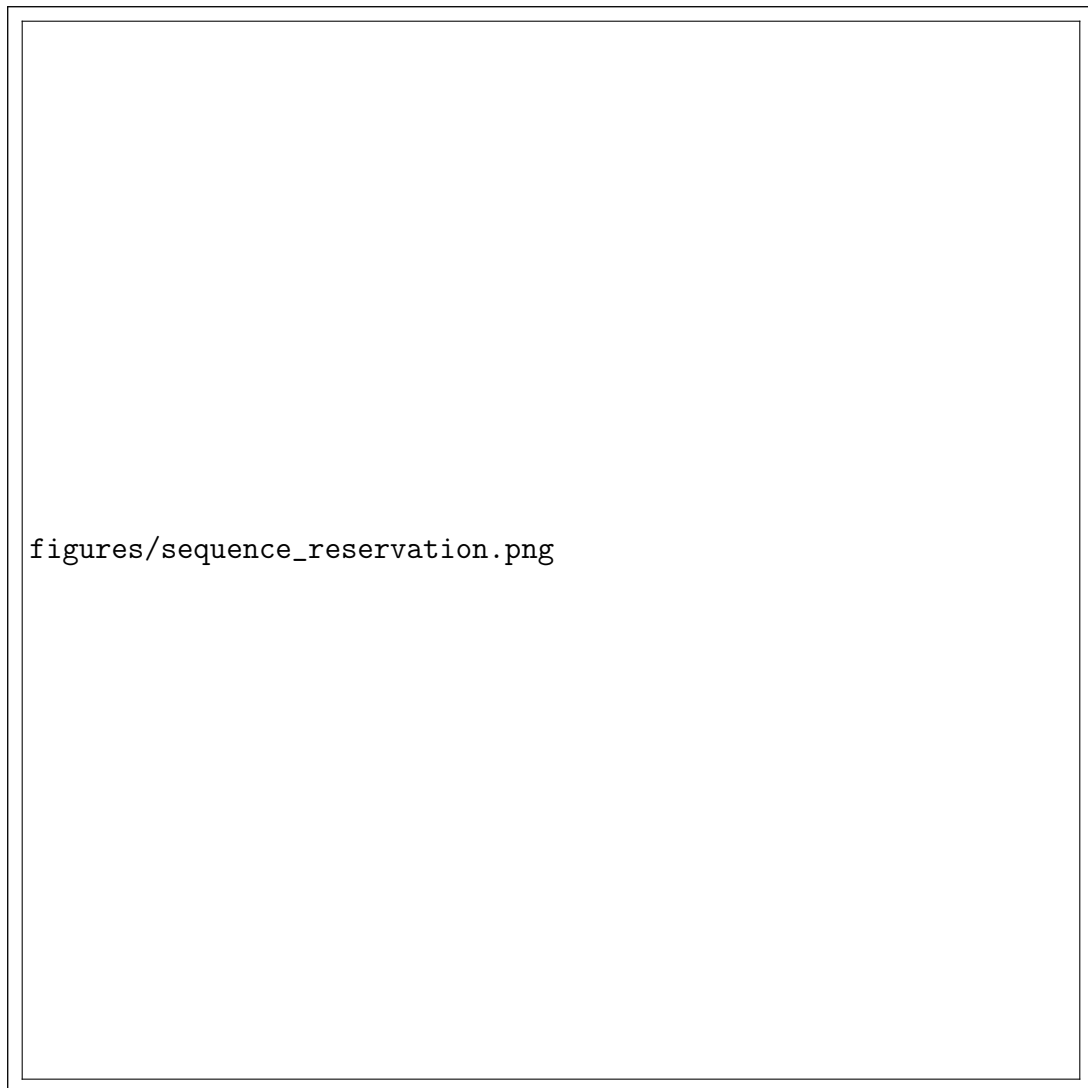


FIGURE 4.3 – Diagramme de séquence — Réservation d’une séance

4.3.5 Diagramme de classes

4.3.5.1 Définition

Le diagramme de classes est une représentation graphique qui illustre la structure statique d’un système en montrant les classes, leurs attributs, méthodes, relations et contraintes. C’est le diagramme central de la conception orientée objet, servant de fondement au modèle de données de l’application.

4.3.5.2 Composition d'un diagramme de classes

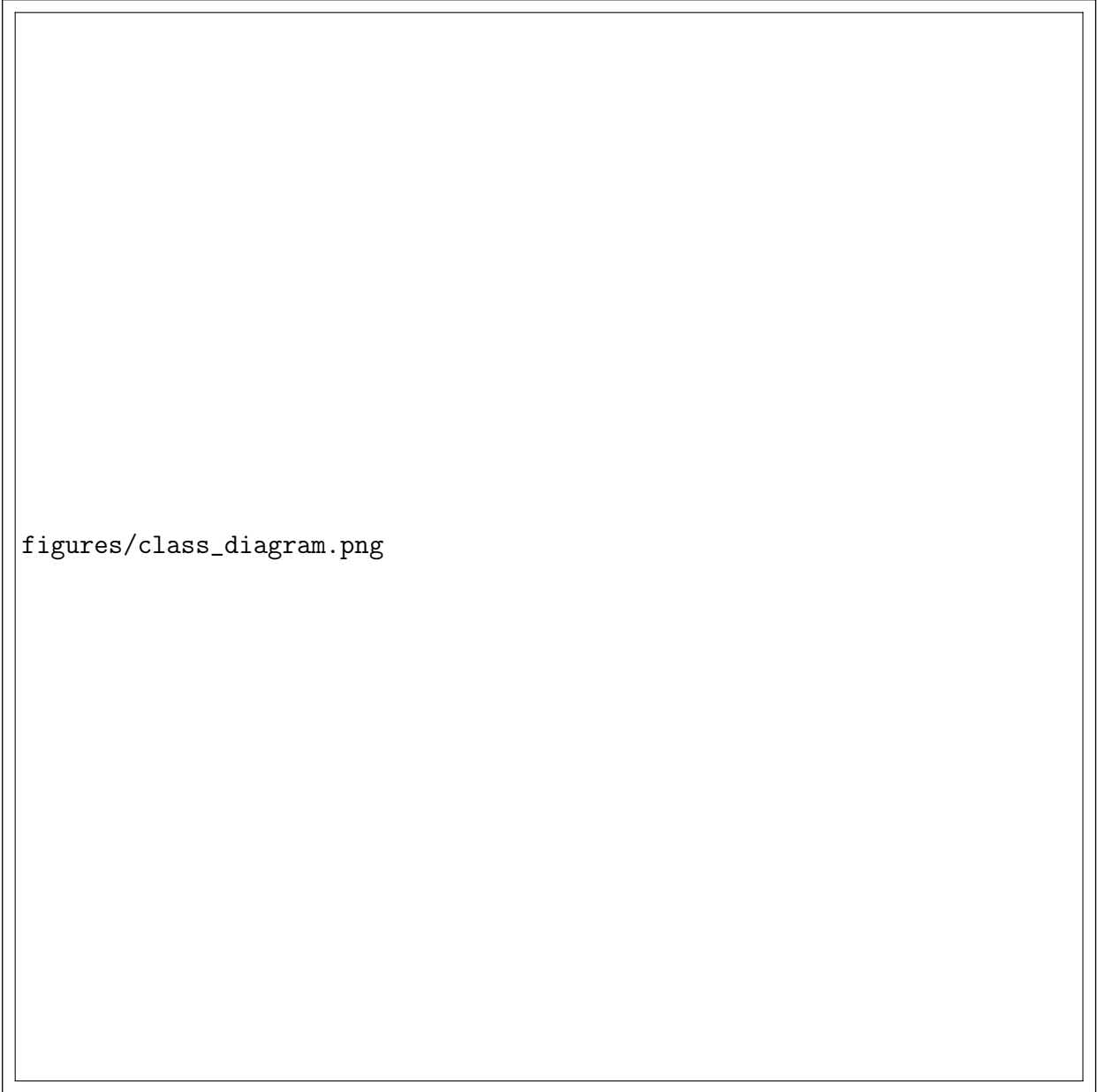
Un diagramme de classes comprend les éléments suivants :

- **Les classes** : représentent des objets partageant une structure et un comportement communs. Chaque classe est composée d'un nom, d'attributs et de méthodes.
- **Les associations** : relations structurelles entre classes décrivant les connexions entre leurs instances (avec multiplicités : 1, 0..*, 1..*).
- **L'héritage** : relation hiérarchique entre une classe parent et ses classes filles, représentée par une flèche à triangle creux.
- **Les énumérations** : types de données à valeurs fixes, représentées avec le stéréotype *«enumeration»*.
- **Les attributs** : données encapsulées dans les objets, avec leur type et leur visibilité (+ public, - privé).

4.3.5.3 Diagramme de classes d'EduMatch

La figure 4.4 présente le diagramme de classes complet d'EduMatch. Le modèle est organisé en sept zones fonctionnelles :

1. **Gestion utilisateurs** : User (classe parent), Etudiant, Professeur et l'énumération RoleEnum.
2. **Profil formateur** : Document avec l'énumération TypeDocument (PHOTO, CV, DIPLOME, CERTIFICAT) et l'énumération StatutValidation.
3. **Référentiel** : hiérarchie Domaine → Niveau → Matière avec l'énumération TypeDomaine (ACADEMIQUE, PROFESSIONNEL).
4. **Matching** : ProfMatiereTarif (table de liaison ternaire) et DemandeMatiere.
5. **Réservations** : Disponibilite, Reservation et Avis.
6. **Messagerie directe** : Conversation et Message.
7. **Chatbot EduBot** : ChatSession et ChatMessage avec l'énumération RoleMessage (USER, ASSISTANT).



figures/class_diagram.png

FIGURE 4.4 – Diagramme de classes complet — EduMatch

4.4 Architecture Technique

4.4.1 Architecture 3-tiers

EduMatch repose sur une architecture **3-tiers** qui sépare clairement les responsabilités entre trois couches indépendantes, conformément aux bonnes pratiques de développement logiciel.

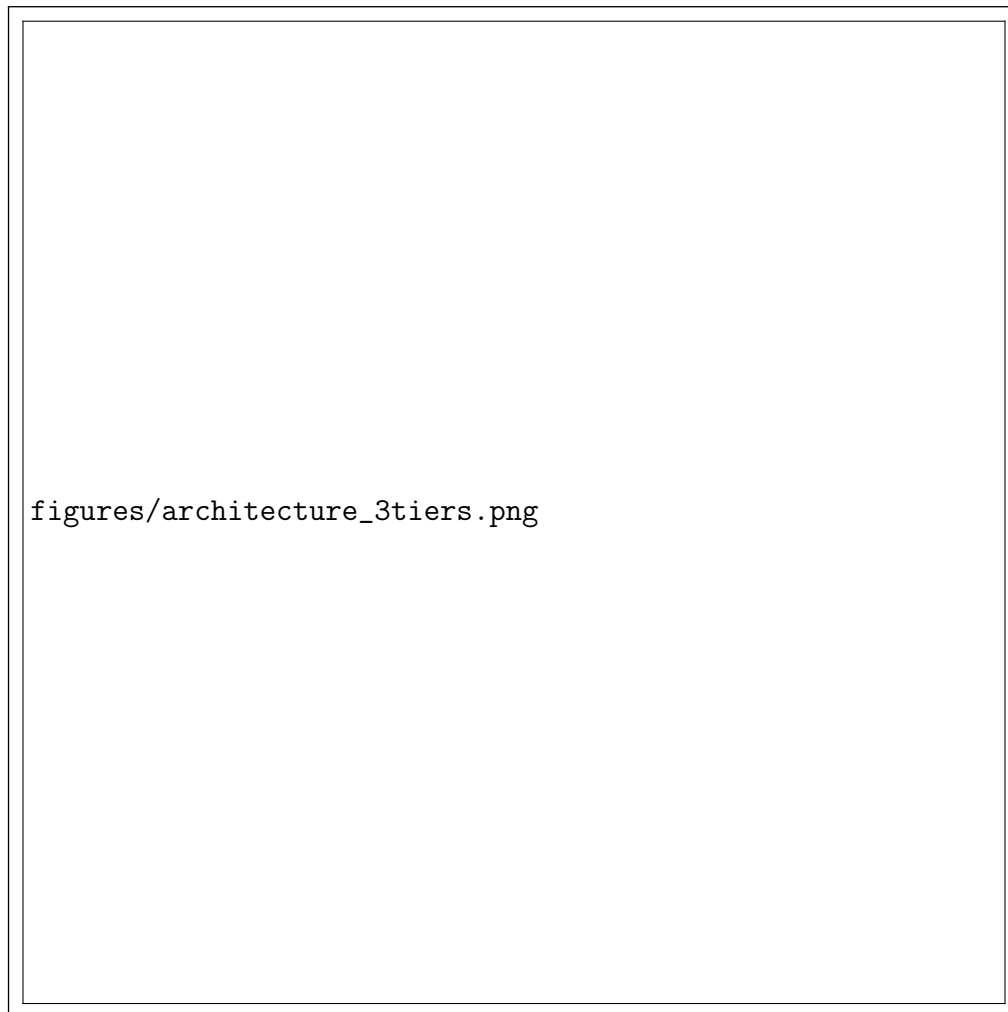


FIGURE 4.5 – Architecture 3-tiers d'EduMatch

- **Couche Présentation (Tier 1)** : application React (Vite) accessible via navigateur. Elle gère l'interface utilisateur (UI), les interactions et la communication avec le backend via des requêtes HTTP/REST sécurisées par token JWT, grâce à la bibliothèque **Axios**.
- **Couche Métier (Tier 2)** : API RESTful développée avec **FastAPI** (Python). Elle contient l'ensemble de la logique applicative : authentification, gestion des profils, réservations, matching et traitement du langage naturel via l'API Groq.
- **Couche Données (Tier 3)** : base de données relationnelle **PostgreSQL**, gérée via l'ORM **SQLAlchemy**. Elle stocke l'ensemble des données : utilisateurs, formateurs, réservations, matières, sessions de chat et messages.

4.4.2 Architecture du chatbot EduBot

EduBot adopte une architecture hybride combinant un flux guidé structuré et un LLM pour offrir une expérience conversationnelle naturelle et intelligente (Figure 4.6).

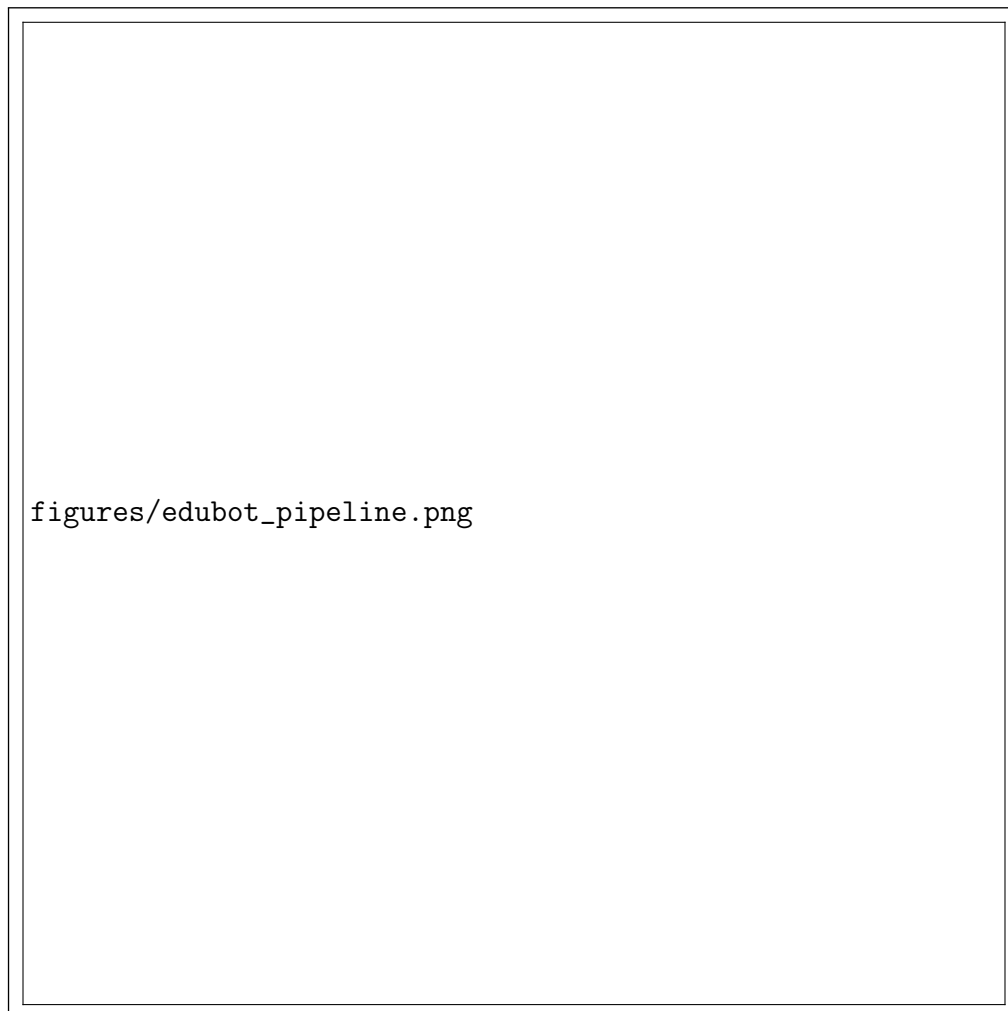


FIGURE 4.6 – Pipeline de traitement d’EduBot

Le pipeline se déroule en quatre étapes séquentielles :

1. **Extraction NLP cumulative** : à chaque message, toute la conversation est soumise à LLaMA 3.1 via l’API Groq. Le modèle retourne un objet JSON structuré contenant les critères identifiés (*domaine_type*, *matiere*, *niveau*, *budget_max*, *mode*, *ville*).
2. **Détection des critères manquants** : le système compare les critères extraits avec la liste des critères requis selon le domaine (académique ou professionnel). Si des critères manquent, EduBot génère une question de suivi naturelle via le LLM.

3. **Moteur de scoring multicritères** : une fois tous les critères collectés, chaque formateur validé reçoit un score de compatibilité sur 100 points selon six dimensions pondérées présentées dans le tableau 4.2.
4. **Génération de réponse naturelle** : EduBot génère une réponse conversationnelle contextualisée via le LLM pour annoncer les résultats, gérer les alternatives ou signaler une ville non couverte.

TABLE 4.2 – Pondération des critères du moteur de matching EduBot

Critère	Poids	Méthode de calcul
Matière	25 pts	Vérification dans <code>prof_matiere_tarif</code> avec variantes lexicales
Niveau	40 pts	Match exact (40), niveau adjacent (12), incompatible (0)
Ville	15 pts	Match exact (15), partiel (7), non applicable si en ligne (0)
Budget	10 pts	Tarif $\leq$ budget (10), $\leq 115\%$ (6), $\leq 130\%$ (3), sinon 0
Mode	5 pts	Mode correspondant ou formateur "les deux" (5), sinon 0
Réputation	5 pts	Note ≥ 4.5 (5), ≥ 4.0 (3), ≥ 3.5 (1)
Total	100 pts	Top 3 formateurs retournés par score décroissant

4.4.3 Workflow de validation des formateurs

EduMatch implémente un workflow de validation en quatre étapes garantissant la qualité des profils disponibles sur la plateforme :

1. **Incomplet** : statut attribué à l'inscription. Le formateur doit compléter son profil (téléphone, bio, tarif, au moins une matière).

2. **En attente** : le formateur clique sur "Soumettre pour validation" une fois le profil complété à 100%. L'administrateur reçoit une notification.
3. **Validé** : l'administrateur approuve le profil. Le formateur devient visible dans les résultats de recherche et dans EduBot.
4. **Refusé** : l'administrateur refuse le profil. Le formateur peut corriger et re-soumettre.

4.5 Conclusion

Dans ce chapitre, nous avons présenté une analyse complète de la plateforme EduMatch en suivant une démarche structurée alliant étude des besoins et conception UML.

Nous avons d'abord identifié les trois acteurs principaux (Étudiant, Formateur, Administrateur) et défini les besoins fonctionnels (recherche, chatbot, réservation, validation, messagerie, analytiques) ainsi que les besoins non fonctionnels (performance, sécurité, scalabilité, ergonomie).

Nous avons ensuite présenté les diagrammes UML de la plateforme : le diagramme de cas d'utilisation global, le diagramme d'activité du flux EduBot, le diagramme de séquence de la réservation, et le diagramme de classes complet en sept zones fonctionnelles.

Enfin, nous avons détaillé l'architecture technique 3-tiers (React / FastAPI / PostgreSQL), le pipeline de traitement d'EduBot et le workflow de validation des formateurs.

Le chapitre suivant sera consacré à la réalisation et à la présentation des interfaces de la plateforme EduMatch.

Un chatbot est un programme informatique capable de simuler une conversation humaine. Il a été programmé pour accomplir une tâche donnée et délivrer une réponse préenregistrée à un moment spécifique[2]. Il peut communiquer en temps réel en utilisant le langage naturel,

CONTEXTE DU PROJET

répondre aux demandes et offrir des services et des solutions adaptés aux besoins des utilisateurs. Les chatbots deviennent de plus en plus populaires, et de nombreuses entreprises les utilisent pour fournir des services clients. Il existe deux catégories de chatbots. Chacune vient avec ses avantages et ses limitations :

- **Les chatbots basés sur des règles** : ou des bots en arbre de décision utilisent des règles prédéterminées pour répondre aux entrées des utilisateurs de manière directe, suivant un flux en forme d'arbre [3]. Ils sont couramment utilisés pour le support client et pour répondre aux questions concernant les produits ou services. Aucune déduction ne sera tirée par le chatbot de ses interactions précédentes. Il est plus simple à utiliser et offre une expérience plus personnalisée et conviviale. L'utilisateur interagit en cliquant sur des boutons, qui le dirigent ensuite vers différents scénarios.

Domino's Pizza est l'un des meilleurs exemples de chatbots basés sur des règles.

- **Les chatbots IA** : Les chatbots basés sur l'IA, également appelés interfaces utilisateur conversationnelles, utilisent l'apprentissage profond et le traitement de langage naturel [3]. Ce sont actuellement les chatbots les plus intelligents, car ils peuvent comprendre l'intention et le contexte de l'utilisateur. En tirant parti du traitement du langage naturel, ils sont capables de lire et de comprendre le langage humain. Les chatbots basés sur l'IA ne sont pas livrés clé en main. Ils ne fournissent pas un modèle général, mais plutôt un modèle spécifique qui doit être entraîné. Ils doivent être pré-configurés avec des exemples de phrases pour chaque scénario. Ils demandent donc plus d'efforts pour la configuration, mais avec le temps, ils offrent une meilleure expérience au client.

Ils peuvent être divisés en deux catégories :

1. **Systèmes de récupération** : Les systèmes de chatbots basés sur la récupération fonctionnent en sélectionnant des réponses préexistantes à partir d'une base de

données, en fonction de la requête de l'utilisateur. Cette méthode repose sur l'analyse de la similarité entre la question posée et les questions stockées dans la base de données, en utilisant des techniques telles que le traitement du langage naturel (NLP) et la compréhension du langage naturel (NLU) pour comprendre l'intention de l'utilisateur.

2. **Systèmes Généraux** : Les systèmes généraux, en revanche, créent des réponses uniques pour chaque requête en temps réel. Ils s'appuient sur des modèles d'apprentissage profond, tels que les réseaux de neurones, pour générer des réponses qui n'ont pas été explicitement programmées dans la base de données. Ces systèmes peuvent apprendre de vastes ensembles de données et sont capables de produire des réponses qui imitent le langage humain de manière convaincante.

Les robots de type IA sont capables de reconnaître un mot mal orthographié et de répondre en conséquence, tandis qu'un chatbot simple ne serait pas incapable de répondre. De plus, ils peuvent proposer des suggestions et notifier les utilisateurs, ce qui contribue à personnaliser l'expérience du chatbot. Les assistants virtuels tels qu'Amazon Alexa, Google Assistant, Siri d'Apple et Microsoft Cortana sont parmi les meilleures illustrations de chatbots basés sur l'IA.

TABLE 4.3 – Chatbots IA VS Chatbots basés sur des règles [4]

	Chatbots IA	Chatbots basés sur des règles
Caractéristiques - S'améliorent au fur et à mesure que de nouvelles données sont disponibles. - Spécifiques à une tâche. - Directs. - Flux en forme d'arbre.	- Acquièrent des connaissances à partir des données collectées. - Règles prédéterminées.	
Avantages - La conversation semble typique et humaine. - Ils peuvent gérer un grand volume de demandes de service client. - Bons pour fournir des réponses prédéfinies aux questions de vos clients.	- Les chatbots IA sont capables d'apprendre et de s'améliorer d'eux-mêmes au fil du temps. - Les chatbots basés sur des règles peuvent être développés plus rapidement et plus facilement comparés aux chatbots utilisant l'IA.	
Limitations - Le contexte d'une conversation peut ne pas être compris, ce qui peut entraîner des malentendus. - Nécessitent d'être fréquemment mis à jour et améliorés. - Incapables de fournir une expérience personnalisée. - Peuvent parfois se bloquer lorsque le client utilise une forme linguistique différente.	- Leur création et leur maintenance sont coûteuses. - Extrêmement utiles pour des interactions linéaires et ne proposent que des réponses issues d'un ensemble de directives définies.	

Chapitre

5

Réalisation, justification des choix et implémentation

Sommaire

5.1	Introduction	10
5.2	Environnement de développement et justification des choix technologiques	10
5.2.1	Environnement de développement	10
5.2.2	Choix du modèle LLM — LLaMA 3.1 via l'API Groq . .	11
5.2.3	Choix de la stratégie d'optimisation IA — Prompt Engineering	12
5.2.4	Choix du framework backend — FastAPI	13
5.2.5	Choix du framework frontend — React avec Vite	14
5.2.6	Choix de la base de données — PostgreSQL	15
5.2.7	Choix de l'algorithme de matching	15
5.3	Présentation des interfaces réalisées	16
5.3.1	Page d'accueil et authentification	16
5.3.2	Espace étudiant	17
5.3.3	Espace formateur	18
5.3.4	Chatbot EduBot	19
5.3.5	Espace administrateur	20
5.4	Implémentation des composants clés	21
5.4.1	Algorithme de matching multicritères	21
5.4.2	Pipeline conversationnel d'EduBot	22
5.4.3	Gestion de l'authentification et des autorisations	23
5.4.4	Composant de recherche et affichage des résultats	23

5.5	Tests et validation	24
5.5.1	Tests unitaires et d'intégration	24
5.5.2	Tests fonctionnels	25

5.1 Introduction

Ce chapitre constitue le cœur opérationnel du rapport. Il est organisé en deux grandes parties complémentaires. La première partie justifie chacun des choix technologiques retenus pour la réalisation d'EduMatch en les comparant objectivement aux alternatives disponibles. La seconde partie présente la réalisation concrète de la plateforme : interfaces utilisateurs, implémentation des composants intelligents, résultats des tests et modalités de déploiement local.

Cette organisation répond à une logique académique rigoureuse : avant de montrer ce qui a été construit, il convient d'expliquer pourquoi chaque brique technologique a été choisie. Ces justifications démontrent la maturité technique et la réflexion critique qui ont présidé aux décisions de conception.

5.2 Environnement de développement et justification des choix technologiques

5.2.1 Environnement de développement

Le développement d'EduMatch a été réalisé sur la configuration matérielle et logicielle suivante.

TABLE 5.1 – Environnement matériel et logiciel de développement

Composant	Détail
Système d'exploitation	Windows 11 (64 bits)
Processeur	Intel Core i5 – 11ème génération, 2,4 GHz
Mémoire vive	16 Go DDR4
Stockage	SSD 512 Go
Éditeur de code	Visual Studio Code 1.88
Gestionnaire de version	Git 2.44 / GitHub
Terminal	PowerShell + Git Bash
Navigateur de test	Google Chrome 124 / Firefox 125

Visual Studio Code a été préféré à des IDE plus lourds comme IntelliJ IDEA ou PyCharm car il offre nativement d'excellentes extensions pour Python, React et la gestion Git, tout en restant léger et réactif. Git a été utilisé avec des commits réguliers et des messages explicites, permettant de tracer l'historique complet du développement et de revenir à tout moment à un état stable antérieur.

5.2.2 Choix du modèle LLM — LLaMA 3.1 via l'API Groq

Le choix du modèle de langage est déterminant pour les performances d'EduBot. Plusieurs alternatives ont été étudiées et comparées selon des critères objectifs.

TABLE 5.2 – Comparaison des modèles LLM pour EduBot

Critère	GPT-4 (OpenAI)	Gemini 1.5 (Google)	LLaMA 3.1 via Groq
Modèle open-source	Non	Non	Oui (Meta AI)
Coût d'utilisation	Élevé	Modéré	Gratuit (Free tier)
Latence d'inférence	500-2000 ms	400-1500 ms	< 200 ms
Qualité NLP	Excellente	Très bonne	Très bonne
Déploiement local	Non	Non	Oui
Indépendance fournisseur	Faible	Faible	Élevée

LLaMA 3.1 via l'API Groq s'impose pour plusieurs raisons décisives. La latence ultra-faible, inférieure à 200 millisecondes, est rendue possible par les puces LPU (Language Processing Unit) spécialisées de Groq, conçues exclusivement pour l'inférence de modèles de langage. Cette performance est indispensable pour EduBot : une réponse perçue comme instantanée par l'utilisateur maintient la fluidité de la conversation et évite la frustration liée aux temps d'attente. Le coût nul en phase de développement est également un argument déterminant dans le contexte d'un projet de fin d'études sans budget alloué aux services cloud. La nature open-source de LLaMA garantit par ailleurs une souveraineté technologique : il est possible de déployer le modèle en local à terme, sans dépendance à un fournisseur propriétaire susceptible de modifier ses tarifs ou ses conditions d'utilisation. Enfin, la qualité du modèle est amplement suffisante pour la tâche d'EduBot, qui consiste essentiellement à extraire six critères structurés depuis une conversation et à générer des réponses contextuelles simples, sans nécessiter les capacités avancées de raisonnement complexe de GPT-4.

5.2.3 Choix de la stratégie d'optimisation IA — Prompt Engineering

Après avoir justifié le choix du modèle LLM, il convient de justifier la stratégie d'optimisation retenue pour l'adapter au cas d'usage spécifique d'EduBot. Le tableau suivant présente une comparaison des trois approches étudiées.

TABLE 5.3 – Comparaison Fine-tuning, RAG et Prompt Engineering pour EduMatch

Critère	Fine-tuning	RAG	Prompt Engineering
Ressources nécessaires	GPU + dataset annoté	Base vectorielle	Aucune infrastructure
Délai de mise en œuvre	Plusieurs semaines	2 à 3 jours	Quelques heures
Adéquation au besoin	Sur-dimensionné	Données non structurées	Idéal (données en base SQL)
Contrôle du format	Difficile	Difficile	Excellent (JSON garanti)
Coût	Élevé	Modéré	Nul
Maintenabilité	Re-entraînement	Mise à jour vecteurs	Modification du prompt

Le Prompt Engineering est retenu pour trois raisons fondamentales. Premièrement, la base de connaissances d'EduMatch est déjà structurée en base de données relationnelle PostgreSQL. Le RAG, qui excelle lorsque les données sont non structurées ou dispersées dans des documents textuels, n'apporterait aucune valeur ajoutée ici : une fois les six critères extraits de la conversation par le LLM, une simple requête SQL suffit pour interroger les formateurs correspondants. Injecter une base vectorielle ajouterait une complexité architecturale sans bénéfice fonctionnel. Deuxièmement, le Prompt Engineering garantit un contrôle total sur le format de sortie du modèle. En spécifiant explicitement dans le prompt que la réponse doit être un objet JSON valide avec exactement six champs nommés, le risque de parsing erroné côté serveur est quasi éliminé. Troisièmement, dans le contexte d'un projet de quatre mois mené en solo, le Prompt Engineering permet une itération rapide : modifier un prompt prend quelques minutes, là où un re-entraînement nécessite des jours et des ressources GPU.

5.2.4 Choix du framework backend — FastAPI

Le backend d'EduMatch devait répondre à plusieurs exigences cumulées : développement rapide, performances asynchrones pour gérer les appels non bloquants à l'API Groq, documentation automatique des endpoints et validation rigoureuse des données entrantes.

TABLE 5.4 – Comparaison des frameworks backend

Critère	Django	Flask	Node.js/Express	FastAPI
Support asynchrone natif	Limité	Limité	Excellent	Excellent
Documentation auto-générée	Non	Non	Non	Swagger/ReDoc intégré
Validation des données	Formulaires	Manuelle	Manuelle	Pydantic natif
Vitesse de développement	Rapide	Lente	Moyenne	Très rapide
Intégration Python IA	Très bonne	Très bonne	Limitée	Excellente
Courbe d'apprentissage	Élevée	Faible	Moyenne	Faible

FastAPI est retenu comme le framework le plus adapté au profil du projet. Son support natif de la programmation asynchrone via `async/await` est indispensable pour les appels non bloquants à l'API Groq : pendant qu'EduBot attend la réponse du LLM, le serveur peut traiter d'autres requêtes simultanément. La génération automatique de la documentation interactive Swagger UI à partir des annotations Python est un atout considérable pour la phase de tests et la qualité professionnelle du livrable. La validation automatique des données via Pydantic, qui vérifie les types et contraintes à chaque entrée API, élimine une classe entière de bugs sans écriture de code de validation supplémentaire. Enfin, l'écosystème Python de FastAPI permet d'intégrer nativement toutes les bibliothèques d'intelligence artificielle et de traitement de données utilisées dans le projet.

5.2.5 Choix du framework frontend — React avec Vite

Pour le développement de l'interface utilisateur, React avec Vite a été préféré à Vue.js et Angular pour plusieurs raisons. React dispose de l'écosystème le plus large parmi les frameworks JavaScript modernes, avec une bibliothèque de composants disponibles pour chaque besoin fonctionnel : calendrier de disponibilités, interface de messagerie, graphiques analytiques et composants de chat. Sa popularité garantit la disponibilité de documentation et de solutions à la plupart des problèmes rencontrés. Angular a été écarté en raison de son architecture rigide et de sa courbe d'apprentissage élevée, incompatibles avec les délais d'un projet de quatre mois mené en solo. Vue.js, bien que léger et élégant, dispose d'un écosystème moins riche que React pour les besoins spécifiques du projet. Vite a été choisi à la place de Create React App pour ses temps de démarrage et de rechargement à chaud remarquablement plus rapides, améliorant significativement la productivité de développement.

Le système de thème dynamique (dark/light mode) a été implémenté via des variables CSS globales, permettant un basculement instantané sans rechargement de page et sans dépendance à une bibliothèque externe de theming.

5.2.6 Choix de la base de données — PostgreSQL

PostgreSQL a été préféré à MySQL, MongoDB et SQLite pour plusieurs raisons complémentaires. EduMatch gère des données fortement relationnelles avec des dépendances complexes : un formateur est lié à des matières, chaque matière est associée à un niveau et un domaine, chaque formateur a des disponibilités liées à des réservations elles-mêmes liées à des avis. PostgreSQL garantit l'intégrité référentielle de ces relations via les clés étrangères et les contraintes, là où MongoDB nécessiterait une gestion manuelle de la cohérence des données. Sur les requêtes de matching qui impliquent plusieurs jointures et filtres combinés, PostgreSQL offre un optimiseur de requêtes mature et performant. SQLite a été écarté car il n'est pas adapté à un environnement multi-utilisateurs concurrent. MySQL, bien que viable, présente des performances inférieures à PostgreSQL sur les requêtes analytiques complexes du tableau de bord administrateur.

5.2.7 Choix de l'algorithme de matching

Trois approches ont été étudiées pour le système de recommandation. Le filtrage collaboratif, qui recommande en se basant sur les préférences d'utilisateurs similaires, est très efficace mais requiert un historique volumineux d'interactions. Cette approche est inadaptée au lancement d'EduMatch, où la base d'utilisateurs est nécessairement limitée. L'approche par embeddings sémantiques, qui vectorise les profils formateurs et les requêtes apprenants pour calculer des similarités cosinus, introduit une complexité technique élevée, une opacité des résultats difficile à expliquer à l'utilisateur, et un besoin d'infrastructure supplémentaire (modèle d'embedding, calcul vectoriel). Le scoring multicritères pondéré a finalement été retenu car il est transparent (le score par critère est visible par l'utilisateur, renforçant la confiance dans le système), opérationnel dès le premier formateur inscrit, facilement ajustable en modifiant les pondérations sans toucher à l'architecture, et son calcul est rapide et linéaire.

La pondération des critères a été définie en s'appuyant sur une analyse des facteurs les plus discriminants dans le choix d'un formateur, telle que reflétée dans les pratiques observées sur le marché tunisien.

TABLE 5.5 – Justification de la pondération des critères de matching

Critère	Poids	Justification
Matière	25 pts	Critère éliminatoire : un formateur qui n'enseigne pas la matière demandée est exclu
Niveau	40 pts	Critère le plus discriminant : enseigner le lycée ne garantit pas la capacité à enseigner
Ville	15 pts	Déterminant pour le présentiel, neutre pour les cours en ligne
Budget	10 pts	Une flexibilité modérée est possible si la qualité du formateur le justifie
Mode	5 pts	Préférence non rédhibitoire, notamment si le formateur accepte les deux modes
Réputation	5 pts	Pondéré modérément pour ne pas pénaliser les nouveaux formateurs sans historique

5.3 Présentation des interfaces réalisées

Cette section présente les principales interfaces de la plateforme EduMatch, organisées par espace utilisateur. L'ensemble de l'interface a été développé avec React et stylisé via un système de variables CSS permettant le basculement dynamique entre le mode clair et le mode sombre. Les interfaces sont accessibles depuis un navigateur web standard sans installation préalable.

5.3.1 Page d'accueil et authentification

La page d'accueil constitue le premier point de contact entre la plateforme et ses visiteurs. Elle est structurée en plusieurs sections : une section héros présentant la proposition de valeur d'EduMatch avec un appel à l'action, une section statistiques dynamiques affichant en temps réel le nombre de formateurs inscrits, d'apprenants actifs et de réservations effectuées, une section de présentation des domaines disponibles (académique et professionnel), et une section de témoignages d'utilisateurs.

[Figure 3.1 – Page d’accueil d’EduMatch – vue desktop]

FIGURE 5.1 – Page d’accueil d’EduMatch

La page d’inscription permet à un nouvel utilisateur de créer un compte en trois étapes : la sélection du rôle (étudiant ou formateur), la saisie des informations personnelles (nom, prénom, adresse email, mot de passe et ville), et l’acceptation des conditions d’utilisation. La validation côté client fournit un retour immédiat à l’utilisateur, tandis que la validation Pydantic côté serveur garantit l’intégrité des données avant leur persistance en base. Les mots de passe sont hachés avec bcrypt avant stockage.

La page de connexion permet l’authentification par email et mot de passe. Lors d’une connexion réussie, l’API FastAPI génère un token JWT signé avec une clé secrète et une durée de validité configurée. Ce token est stocké dans le localStorage du navigateur côté client et transmis dans l’en-tête Authorization de toutes les requêtes ultérieures vers les endpoints protégés. Le frontend React gère automatiquement l’expiration du token et redirige l’utilisateur vers la page de connexion lorsque celui-ci expire.

5.3.2 Espace étudiant

Après connexion, l’étudiant est redirigé vers son tableau de bord personnalisé. Celui-ci présente une vue synthétique de son activité : les réservations à venir avec leurs dates et statuts, les formateurs sauvegardés en favoris, les messages non lus avec les formateurs et les dernières interactions avec EduBot. Des suggestions de formateurs populaires dans les domaines récemment recherchés complètent cette vue d’ensemble.

La page de recherche est l’une des fonctionnalités les plus utilisées de la plateforme. Elle propose un panneau de filtres latéral comprenant des champs de sélection pour le domaine, la matière, le niveau académique, la ville, le budget horaire maximum et le mode d’enseignement. Les résultats s’affichent en temps réel sous forme de cartes formateurs triées par note moyenne décroissante. Chaque carte affiche la photo de profil, le nom, la note moyenne avec le nombre d’avis, les matières et niveaux enseignés, les tarifs pratiqués, la ville et la disponibilité prochaine.

La page de profil détaillé d'un formateur s'ouvre en cliquant sur sa carte depuis les résultats de recherche. Elle présente l'intégralité des informations publiques du formateur : biographie complète, photo de profil, liste détaillée des matières et niveaux enseignés avec les tarifs associés, calendrier des créneaux disponibles, ensemble des avis déposés par les apprenants avec leurs notes et commentaires, et un bouton de réservation directe.

La page de réservation guide l'étudiant en trois étapes successives. La première étape consiste à sélectionner un créneau disponible dans le calendrier du formateur ; les créneaux déjà occupés sont grisés et non sélectionnables. La seconde étape permet de confirmer la modalité (en ligne ou en présentiel si le formateur propose les deux) et d'ajouter un message optionnel au formateur. La troisième étape affiche un récapitulatif avant la validation définitive. Pour les séances en ligne, le lien Google Meet est généré immédiatement après confirmation et transmis par notification à l'étudiant et au formateur.

Après la réalisation d'une séance, un formulaire d'évaluation est rendu accessible à l'étudiant. Il peut y attribuer une note de une à cinq étoiles en cliquant sur les étoiles interactives, et rédiger un commentaire textuel libre décrivant son expérience avec le formateur. La soumission de l'avis déclenche automatiquement le recalcul de la note moyenne du formateur en base de données, qui est immédiatement prise en compte dans les calculs de scoring de matching.

5.3.3 Espace formateur

Le tableau de bord du formateur lui offre une vision synthétique de son activité sur la plateforme. Il présente le taux de complétion du profil sous forme de barre de progression avec les éléments manquants listés explicitement, le nombre de réservations en attente de confirmation, le nombre de séances confirmées pour les sept prochains jours, la note moyenne actuelle basée sur tous les avis reçus, et le nombre de messages non lus. Un indicateur visuel du statut de validation (incomplet, en attente, validé, refusé) est mis en évidence avec les actions disponibles correspondantes.

La page de gestion du profil est organisée en sections thématiques : informations personnelles (nom, prénom, photo de profil, biographie, ville, téléphone), paramètres pédagogiques (tarifs en ligne et en présentiel, mode d'enseignement privilégié), matières enseignées avec un système d'ajout dynamique permettant d'associer à chaque matière les niveaux correspondants et le tarif spécifique, et documents justificatifs avec des zones de dépôt pour la photo de profil, le CV, les diplômes et les certificats professionnels.

La gestion des disponibilités s'appuie sur une interface calendrier mensuelle. Le formateur crée ses créneaux en cliquant sur les cellules du calendrier et en renseignant les horaires, le mode proposé et le nombre de places disponibles pour les sessions en groupe. Les créneaux confirmés par des apprenants apparaissent en lecture seule avec le nom de l'apprenant concerné. Le formateur peut annuler un créneau non encore réservé et ajuster le nombre de places d'un créneau existant.

Lorsque le profil atteint un taux de complétion de 100 %, le bouton de soumission pour validation se débloque et passe d'un état grisé inactif à un état actif coloré. Un message d'information indique au formateur que sa demande sera traitée dans un délai de 24 à 48 heures. Après soumission, le statut passe à « En attente » et le bouton est remplacé par un message informatif indiquant que le dossier est en cours d'examen.

5.3.4 Chatbot EduBot

EduBot est accessible depuis toutes les pages de la plateforme via un bouton flottant positionné en bas à droite de l'écran. Un clic sur ce bouton ouvre un panneau conversationnel en superposition sur la page courante. L'interface de conversation distingue visuellement les messages de l'apprenant, alignés à droite sur fond coloré, des réponses d'EduBot, alignées à gauche sur fond neutre avec l'icône du chatbot. Un indicateur animé de trois points clignotants signale que le modèle génère sa réponse, maintenant l'engagement de l'utilisateur pendant le délai d'inférence.

Lors d'une session de recherche, EduBot commence par accueillir l'apprenant et lui demande de décrire son besoin en langage naturel. L'apprenant peut formuler sa demande

librement — « je cherche un prof de maths pour la terminale à Sfax avec un budget de 30 dinars par heure » — et EduBot extrait automatiquement les critères présents grâce au Prompt Engineering. Il pose ensuite des questions ciblées et naturelles pour les critères manquants.

Une fois tous les critères collectés, EduBot présente les résultats sous forme de cartes structurées intégrées directement dans la conversation. Chaque carte affiche le nom et la photo du formateur, son score global sur 100 avec le label de qualité correspondant, le détail du score par critère sous forme de barres de progression colorées, ses tarifs, ses prochains créneaux disponibles et un bouton de réservation directe. Si aucun formateur ne correspond exactement aux critères, EduBot présente un message explicatif et propose automatiquement deux ou trois matières alternatives pour lesquelles des formateurs sont disponibles avec les mêmes paramètres.

5.3.5 Espace administrateur

Le tableau de bord analytique de l'administrateur présente une vue d'ensemble en temps réel de la santé de la plateforme. La ligne supérieure affiche les indicateurs clés sous forme de cartes métriques : nombre total d'utilisateurs inscrits, nombre de formateurs au statut validé, nombre de réservations effectuées sur les 30 derniers jours, et nombre de demandes en attente de traitement. Des graphiques interactifs illustrent l'évolution mensuelle des réservations sur les douze derniers mois, la répartition des formateurs par domaine (académique versus professionnel), et le classement des dix matières les plus demandées par les apprenants.

La page de validation des formateurs liste tous les dossiers en attente avec leur date de soumission et un indicateur de priorité basé sur le délai d'attente. En cliquant sur un dossier, l'administrateur accède à la vue complète du profil : informations personnelles, biographie, matières et niveaux déclarés, tarifs, et documents téléchargés (CV, diplômes, certificats) consultables directement dans le navigateur. Deux boutons d'action permettent

d'approuver ou de refuser le dossier. En cas de refus, un champ texte obligatoire permet de saisir le motif qui sera transmis au formateur par notification.

La page de gestion du référentiel présente la hiérarchie complète Domaine → Niveau → Matière sous forme d'arborescence interactive. L'administrateur peut ajouter, modifier ou désactiver des entrées à chaque niveau de la hiérarchie. Une section dédiée liste les demandes de nouvelles matières soumises par les formateurs, avec pour chacune le nom de la matière proposée, le formateur demandeur, le domaine et le niveau cible, et les boutons d'approbation ou de rejet.

5.4 Implémentation des composants clés

5.4.1 Algorithme de matching multicritères

L'algorithme de matching constitue le cœur intelligent de la plateforme. Implémenté dans le service FastAPI sous la forme d'une fonction Python, il reçoit en entrée les critères extraits par EduBot et la liste des formateurs validés en base, et retourne pour chacun d'eux un dictionnaire contenant le score global, le détail par critère et le label de qualité correspondant.

La logique centrale suit le même ordre de traitement pour chaque formateur. Le critère de la matière est traité en premier car il est éliminatoire : si le formateur n'enseigne pas la matière demandée, sa note globale est immédiatement fixée à zéro et il est exclu des résultats. Le critère du niveau est ensuite évalué avec un système de paliers : une correspondance exacte apporte 40 points, un niveau adjacent (par exemple, un formateur enseignant le lycée pour un apprenant de terminale demandant le « supérieur ») apporte 12 points, et une incompatibilité totale n'apporte aucun point. Le critère de la ville n'est évalué que pour les demandes en présentiel : une correspondance exacte apporte 15 points, une correspondance partielle (par exemple, « Grand Sfax » pour une demande sur « Sfax ») apporte 7 points. Le critère du budget utilise un système de paliers progressifs pour intégrer une flexibilité raisonnée : un tarif strictement inférieur au budget maximum

apporte 10 points, un tarif entre le budget et 115 % du budget apporte 6 points, entre 115 % et 130 % apporte 3 points, et au-delà aucun point. Le critère du mode apporte 5 points si le mode proposé par le formateur correspond exactement à la demande, ou si le formateur accepte les deux modes. Enfin, le critère de réputation apporte de 1 à 5 points selon la note moyenne des avis reçus par le formateur, avec une pondération volontairement faible pour ne pas pénaliser les nouveaux formateurs n'ayant pas encore d'historique.

À l'issue du calcul, les formateurs sont triés par score décroissant. Les trois premiers sont sélectionnés et enrichis avec leurs prochains créneaux disponibles avant d'être retournés à EduBot pour présentation à l'apprenant.

5.4.2 Pipeline conversationnel d'EduBot

Le pipeline conversationnel d'EduBot est orchestré par un service FastAPI dédié qui gère l'état de chaque session de conversation. Chaque session est persistée en base de données sous la forme d'une suite de messages (`ChatSession` → `ChatMessage`), ce qui permet de reprendre une conversation interrompue sans perte du contexte.

À chaque message envoyé par l'apprenant, le service récupère l'intégralité de l'historique de la session depuis la base de données, y ajoute le nouveau message, et soumet le tout au modèle LLaMA 3.1 via l'API Groq avec un prompt système d'extraction précis. Ce prompt instruit le modèle de retourner exclusivement un objet JSON valide sans texte supplémentaire, garantissant ainsi une sortie directement parsable par le backend. La température est fixée à 0,1 pour maximiser le déterminisme de l'extraction. La réponse JSON est parsée, et les critères extraits sont comparés à la liste des critères requis pour le domaine identifié. Si tous les critères sont présents et non nuls, le moteur de matching est déclenché. Sinon, un second appel au LLM — avec cette fois une température plus élevée pour plus de naturel — génère la question de suivi à poser à l'apprenant. La réponse finale est sauvegardée en base et retournée au frontend pour affichage dans l'interface de chat.

5.4.3 Gestion de l'authentification et des autorisations

La sécurité d'EduMatch repose sur une chaîne d'authentification et d'autorisation cohérente. À l'inscription, le mot de passe fourni par l'utilisateur est haché avec l'algorithme bcrypt avant toute persistance en base. bcrypt est choisi pour sa résistance aux attaques par force brute grâce à son facteur de coût ajustable.

Lors de la connexion, FastAPI vérifie le hash du mot de passe fourni contre celui stocké en base via bcrypt. En cas de succès, un token JWT est généré en signant un payload contenant l'identifiant de l'utilisateur, son rôle et la date d'expiration, avec une clé secrète stockée dans les variables d'environnement. Ce token est retourné au frontend qui le stocke et le transmet dans l'en-tête Authorization de toutes les requêtes ultérieures.

Côté FastAPI, un décorateur de dépendance vérifie automatiquement la validité du token sur chaque endpoint protégé : signature cryptographique, expiration et cohérence du rôle avec les droits requis par l'endpoint. Un endpoint accessible uniquement aux administrateurs retournera une erreur 403 si le token présenté correspond à un étudiant ou un formateur.

5.4.4 Composant de recherche et affichage des résultats

L'interface de recherche de formateurs illustre l'intégration entre le frontend React et le backend FastAPI. Le composant de recherche maintient dans son état local React (via le hook useState) les valeurs courantes de l'ensemble des filtres. Chaque modification d'un filtre met à jour l'état local et déclenche optionnellement une mise à jour automatique des résultats. Lorsque l'utilisateur déclenche la recherche, les filtres non vides sont collectés et transmis comme paramètres de requête à l'endpoint de recherche de l'API via Axios. Le composant gère les états de chargement (affichage d'un indicateur visuel pendant la requête), de succès (affichage des résultats sous forme de grille de cartes) et d'erreur (affichage d'un message explicatif avec suggestion de réessayer). Les résultats sont affichés sous forme de composants FormateurCard réutilisables, chacun recevant les données du formateur comme propriétés React.

5.5 Tests et validation

5.5.1 Tests unitaires et d'intégration

La validation technique de la plateforme EduMatch repose sur une suite de tests automatisés réalisée avec le framework pytest pour le backend FastAPI. Deux catégories de tests ont été développées.

Les tests unitaires valident chaque fonction critique de manière isolée, en s'appuyant sur des données de test représentatives couvrant les cas nominaux et les cas limites. L'algorithme de matching a fait l'objet d'une attention particulière : des formateurs fictifs avec des profils variés ont été créés pour tester chaque critère individuellement et en combinaison, notamment les cas de niveau adjacent, de budget légèrement dépassé et de correspondance partielle de ville.

Les tests d'intégration simulent des requêtes HTTP complètes vers les endpoints de l'API via le client de test fourni par FastAPI, qui s'appuie sur HTTPX. Ces tests vérifient les codes de statut HTTP retournés, la structure et le contenu des réponses JSON, et la cohérence des données persistées en base après chaque opération.

TABLE 5.6 – Résultats des tests unitaires et d'intégration

Module testé	Nombre de tests	Résultat	Couverture
Algorithme de matching	12	12/12 PASSED	94 %
Extraction NLP — EduBot	8	8/8 PASSED	87 %
Authentification JWT	6	6/6 PASSED	96 %
Gestion des réservations	10	10/10 PASSED	91 %
Workflow de validation	7	7/7 PASSED	89 %
Total	43	43/43 PASSED	91 % (moy.)

5.5.2 Tests fonctionnels

Les tests fonctionnels ont été conduits manuellement en suivant des scénarios d'utilisation représentatifs couvrant les parcours critiques de chaque acteur. Ces scénarios ont été définis à partir des cas d'utilisation identifiés au chapitre précédent et testés sur l'environnement de développement local avec des données de test réalistes.

TABLE 5.7 – Résultats des tests fonctionnels

Scénario testé	Acteur	Résultat attendu	Statut
Inscription et connexion d'un étudiant	Étudiant	Token JWT généré, redirection	Conforme
Recherche multicritères	Étudiant	Liste filtrée des formateurs	Conforme
Réservation d'une séance en ligne	Étudiant	Créneau réservé, lien Meet généré	Conforme
Dialogue EduBot complet	Étudiant	Top 3 formateurs avec scores	Conforme
EduBot : aucun formateur trouvé	Étudiant	Proposition matières alternatives	Conforme
Complétion du profil à 100 %	Formateur	Bouton de soumission débloqué	Conforme
Soumission et validation du profil	Administrateur	Statut « Validé », visible	Conforme
Ajout d'une nouvelle matière	Administrateur	Matière disponible dans les filtres	Conforme
Dépôt d'un avis après séance	Étudiant	Note moyenne recalculée	Conforme
Basculement dark/light mode	Tous	Thème appliqué instantanément	Conforme

L'ensemble des scénarios testés a produit les résultats attendus sans anomalie détectée. Les quelques observations relevées en cours de test (comportements marginaux sur des formulations atypiques soumises à EduBot) ont été corrigées par ajustement du prompt d'extraction avant la campagne de test finale.